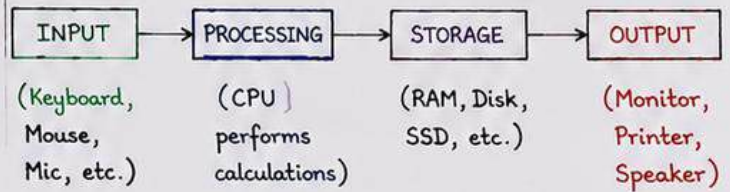


Page 1 : INTRODUCTION TO COMPUTER SYSTEM (FOUNDATION BEFORE OS)

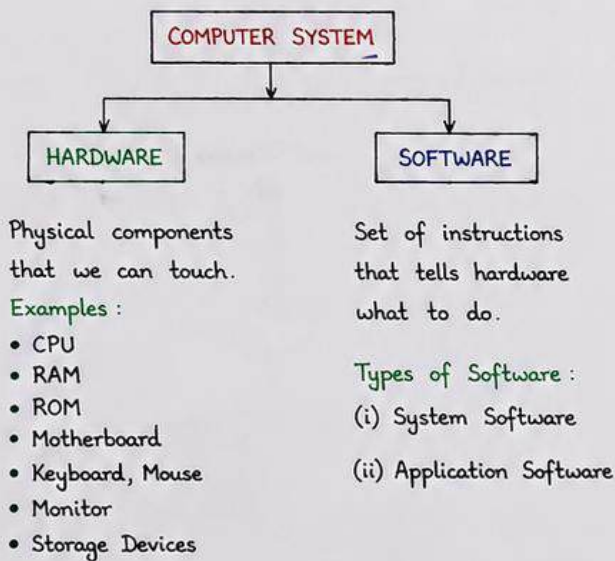
1. What is a Computer ?

A computer is an electronic device that accepts data as input, processes it according to instructions, stores it, and produces meaningful output.

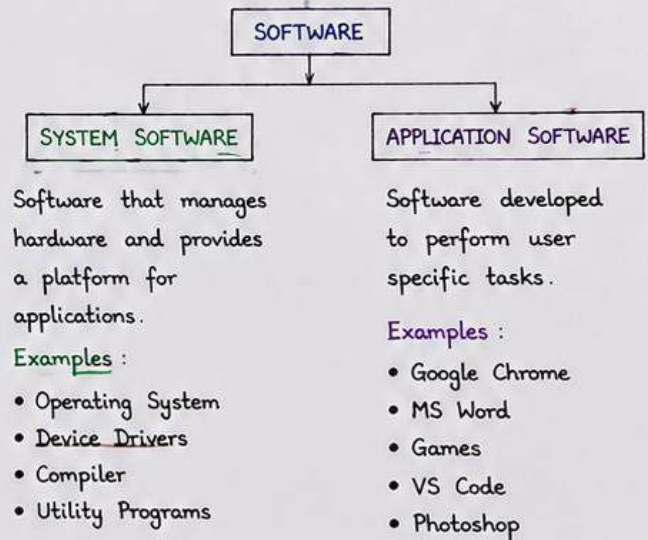
2. Basic Working of Computer



3. Components of Computer System



4. Types of Software

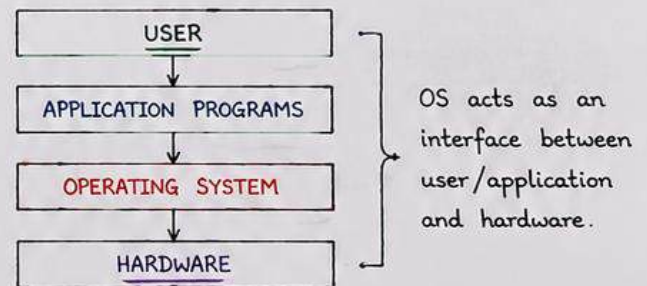


5. Need of Operating System

Hardware alone cannot work efficiently. Without an OS, we face many problems.

- Who controls CPU usage ?
 - Who manages memory ?
 - Who manages files and data ?
 - Who communicates with I/O devices ?
- The solution is OPERATING SYSTEM

6. Position of Operating System



7. Definition of Operating System

(i) Simple Definition :

An Operating System is system software that acts as an interface between the user and computer hardware.

(ii) Technical Definition :

An Operating System is a resource manager, that manages CPU, memory, files and I/O devices while providing services to application programs.

8. Main Goals of Operating System

1. Convenience : Makes computer easy to use.
e.g. Using icons instead of machine commands.
2. Efficiency : Uses hardware resources effectively.
e.g. Keeping CPU busy and reducing idle time.
3. Ability to Evolve : Allows support for new hardware, software and technologies.

Remember :

- Hardware provides power,
- Software provides instructions,
- Operating System connects both and makes everything work together.

Viva Questions :

- Q1. What is an Operating System ?
- Q2. Why is OS called a Resource Manager ?
- Q3. Is OS hardware or software ?

OS is the Manager of Computer System 😊

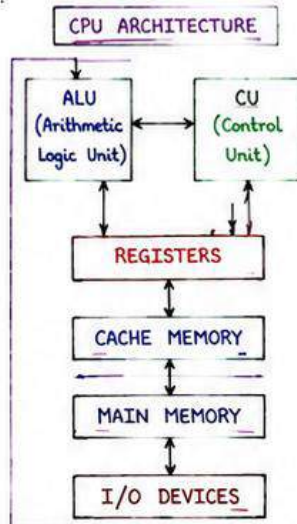
Page 2 : Inside a Computer - CPU, Memory, RAM, ROM, Cache and Storage Hierarchy

1. CPU (Central Processing Unit)

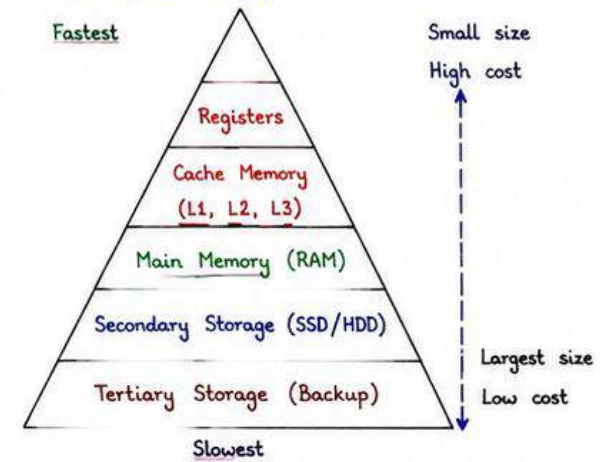
CPU is called the "Brain" of the computer. It performs all the instructions of a program.

Main Parts of CPU :

- ① **ALU** (Arithmetic Logic Unit) - Performs arithmetic and logical operations.
- ② **CU** (Control Unit) - Controls all the operations and flow of data.
- ③ **Registers** - Very small and fast memory inside CPU to store data/instructions.
- ④ **Cache Memory** - High speed memory which stores frequently used data.
- ⑤ **Clock** - Provides timing and synchronization.



2. Memory Hierarchy



Why Memory Hierarchy ?

- To provide high speed (performance)
- To provide large storage capacity
- To reduce cost per bit

3. Primary Memory

(i) RAM (Random Access Memory)

- Volatile (loses data when power off)
- Read/Write memory
- Used to store programs and data that are currently in use.

Types of RAM :

- ① **SRAM** (Static RAM) - Very fast, expensive, used in cache.
- ② **DRAM** (Dynamic RAM) - Slower than SRAM, needs refresh, used in main memory.
- ③ **SDRAM** (Synchronous DRAM)
- ④ **DDR SDRAM** - DDR, DDR2, DDR3, DDR4, DDR5 (successive generations) with higher speed and lower power.

(ii) ROM (Read Only Memory)

- Non-volatile (data remains even when power is off)
- Read Only memory.
- Stores permanent instructions required to boot the system.

Types of ROM :

- ① **PROM** - Programmable ROM
- ② **EPROM** - Erasable PROM
- ③ **EEPROM** - Electrically Erasable PROM
- ④ **Flash Memory** - Fast, used in phones, SSD, pen drives, BIOS.

4. Cache Memory

- Very small but very fast memory.
- Placed between CPU and Main Memory.
- Stores frequently accessed data/instructions.

Types :

- ① **L1 Cache** - Smallest, fastest, inside CPU core.
- ② **L2 Cache** - Larger, slightly slower.
- ③ **L3 Cache** - Largest, shared among cores.

5. Secondary Storage

- Non-volatile, large capacity, slower than main memory.
- Stores data permanently.
- Examples : **SSD, HDD, CD/DVD, Blu-ray, Memory Card.**

Feature	SSD (Solid State Drive)	HDD (Hard Disk Drive)
Technology	Uses NAND Flash	Uses Magnetic Disks
Speed	Very Fast	Relatively Slow
Moving Parts	No	Yes
Power Consumption	Low	High
Cost	High	Low
Durability	More (Shock Resistant)	Less
Noise	Silent	Produces Noise

6. Tertiary / Backup Storage

- Used for long term data storage and backup.
- Examples : Magnetic Tape, Cloud Storage, External Hard Disk, Optical Disks.

Key Takeaways

- ★ CPU is the brain of computer.
- ★ Memory hierarchy gives speed + size + cost balance.
- ★ RAM is volatile, ROM is non-volatile.
- ★ Cache is faster than RAM.
- ★ SSD is faster and better than HDD.
- ★ OS manages all these resources efficiently.



Page 3 : Evolution of Operating Systems & Types of Operating Systems (Part 1)

1. Evolution of Operating Systems

OS evolved with the advancement of hardware, software and user requirements.

- ① • No OS / Serial Processing
User interacts directly with hardware.
- ② • Simple Batch System
Jobs collected and executed in batches.
- ③ • Multiprogramming System
Multiple programs loaded in memory, CPU utilized efficiently.
- ④ • Time Sharing System
Multiple users interact simultaneously through terminals.
- ⑤ • Personal Computer Systems
Single user systems, user friendly OS.
- ⑥ • Network Operating Systems
Resource sharing over a network.
- ⑦ • Distributed Operating Systems
Multiple systems work together and appear as a single system.
- ⑧ • Mobile Operating Systems
For smart phones and tablets.
- ⑧ • Modern / Cloud Operating Systems
Virtualization, cloud computing, AI enabled.

2. Types of Operating Systems

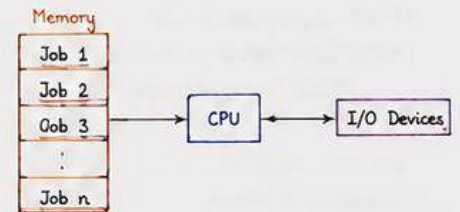
(i) Simple Batch System

- Jobs are collected and executed one by one.
- Uses Job Queue.
- No user interaction during execution.
- Example : Early IBM systems.



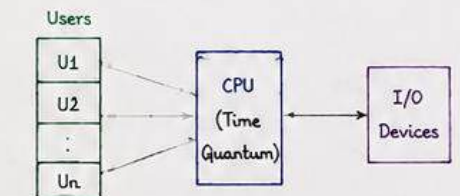
(ii) Multiprogramming System

- Several programs kept in memory.
- When one program waits for I/O, CPU switches to another.
- Improves CPU utilization.
- Example : Early OS/360 systems.



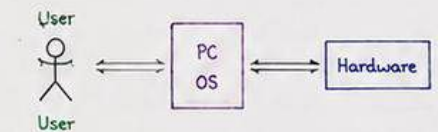
(iii) Time Sharing System

- Extension of multiprogramming.
- CPU time is divided into small time slices (quantum).
- Provides interactive environment.
- Example : UNIX, Multics.



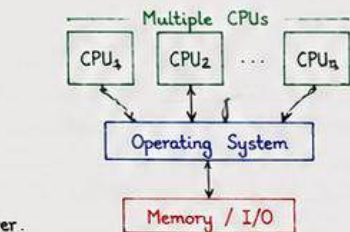
(iv) Personal Computer Systems

- Designed for single user.
- Focus on usability and performance.
- Examples : Windows, macOS, Linux (Desktop).



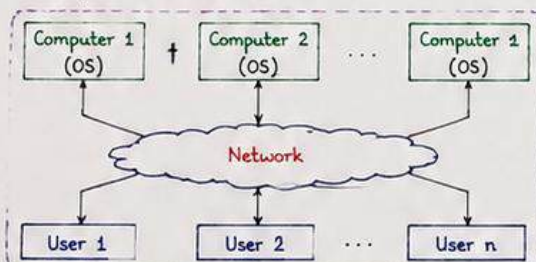
(v) Parallel Operating Systems

- OS manages multiple CPUs working in parallel.
- Two types :
 - Symmetric Multiprocessing (SMP)
 - Asymmetric Multiprocessing (AMP)
- Example : Linux (SMP), Windows Server.



3. Distributed Operating Systems

- Several independent computers work together.
- Users feel it as a single system.
- Provides resource sharing, load balancing, fault tolerance.
- Examples : Amoeba, Plan 9.



4. Real-Time Operating Systems (RTOS)

- Designed to respond to events within strict time limits.
- Two types :

Hard Real-Time System	Soft Real-Time System
- Missing deadline is unacceptable. - Used in critical applications. - Examples : - VxWorks - QNX - RTLinux	- Missing deadline is tolerable. - Used in multimedia, online gaming, etc. - Examples : - Windows - Linux (General Purpose)

★ Key Points to Remember

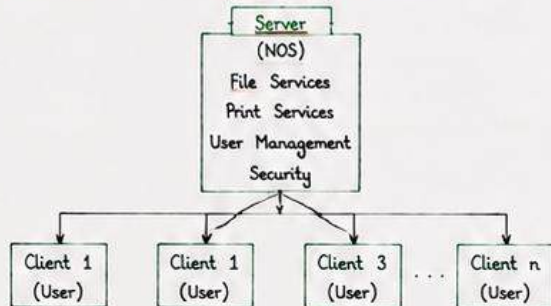
- OS evolved to improve CPU utilization, user interaction and resource sharing.
- Time sharing provides quick response to users.
- Distributed OS provides transparency and resource sharing.
- Real-Time OS is used where timing is more important than throughput.



Page 4 : Evolution of Operating Systems & Types of Operating Systems (Part 2)

5. Network Operating Systems

- OS runs on a server.
- Provides services to multiple systems over a network.
- Examples : Windows Server, Linux Server.

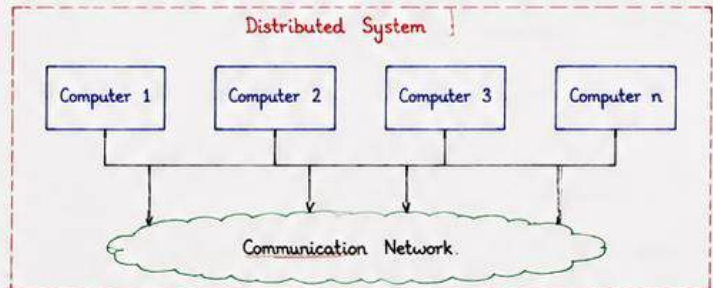


Features :

- Centralized management.
- Resource sharing (files, printers, applications).
- Security and user authentication.
- Backup and data consistency.

6. Distributed Operating Systems

- Multiple computers work together as a single system.
- Users do not know the exact location of resources.
- Goal : Transparency, reliability, high performance.



Characteristics :

- ① Resource sharing
- ② Distribution transparency
- ③ High reliability
- ④ Scalability

7. Mobile Operating Systems

- OS designed for smart phones, tablets, wearable devices.
- Touch based interface, power efficient.
- Examples : Android, iOS.

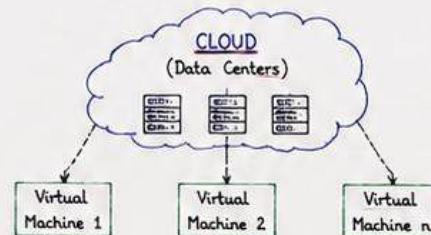
Features :

- ① Touch interface
- ② Lightweight and power efficient.
- ③ Wireless connectivity support
- ④ App management
- ⑤ Security and permission control



8. Modern / Cloud Operating Systems

- Supports virtualization, cloud computing, AI, containers.
- Runs on data centers and cloud platforms.
- Examples : AWS, Google Cloud OS, Azure.



Advantages :

- Scalability
- High availability
- Resource optimization
- Pay-as-you-go model

9. Comparison Table

Type	Main Focus	Runs On	Users	Example
Simple Batch	Job execution	Single Computer	One at a time	Early IBM OS
Multiprogramming	CPU utilization	Single Computer	Multiple	Early OS/360
Time Sharing	User interaction	Single Computer	Many (online)	Multics
Personal	Single user	Personal Computer	One	Windows, macOS
Parallel	Speed (multiple CPUs)	Multi CPU System	Multiple	Linux (SMP)
Network	Resource sharing	Server	Many over network	Windows Server
Distributed	Transparency, sharing	Multiple Systems	Many	Amoeba, Plan 9
Mobile	Mobile devices	Smartphones/Tablets	One	Android, iOS
Cloud	Virtualization, AI	Cloud Infrastructure	Many	AWS, GCP

10. Summary

- ★ OS evolved with the growth of hardware, software and user needs.
- ★ From simple batch systems to cloud platforms, OS has become smarter, faster and more user friendly.
- ★ Different types of OS are designed to solve specific problems.
- ★ Main aim of every OS is to manage resources efficiently and provide better services to users.



Viva Questions :

- Q1. What are the different types of Operating Systems ?
- Q2. What is the difference between Network OS and Distributed OS ?
- Q3. Why is Time Sharing OS called interactive OS ?

Every OS exists to make computing better for us!



Page 5 : Architecture of Operating System & OS Services

1. Architecture of Operating System

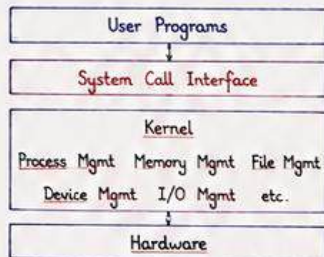
The structure of an OS decides how it is designed and how its components interact with each other.

Main Architectures :

(i) Monolithic Kernel Architecture

- All OS services run in kernel mode.
- Everything is part of a single large program.
- Fast but difficult to maintain.

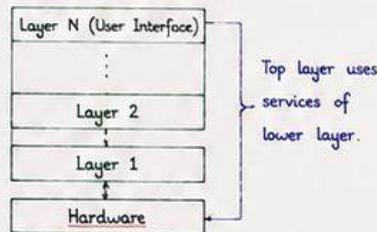
Example : UNIX (Early), Linux.



(ii) Layered Architecture

- OS is divided into layers (levels).
- Each layer uses services of lower layer.
- Easier to design and debug.

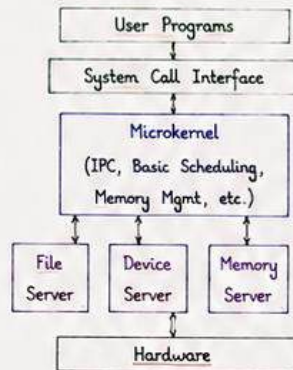
Example : THE (Educational OS).



(iii) Microkernel Architecture

- Only essential services run in kernel.
- Other services run in user space as separate servers.
- More reliable and secure.

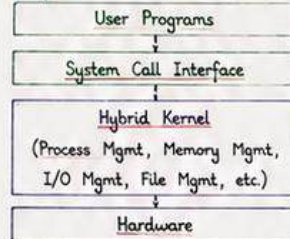
Example : MINIX, QNX.



(iv) Hybrid Kernel Architecture

- Combines advantages of Monolithic and Microkernel.
- Most modern OS follow this.

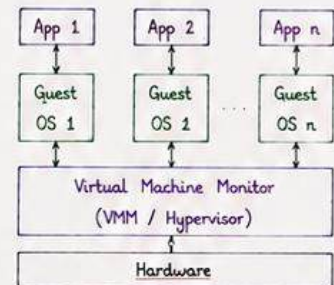
Example : Windows, macOS.



(v) Virtual Machine Architecture

- OS runs as a virtual machine.
- Provides complete hardware abstraction.
- Used in cloud and virtualization.

Example : VMware, VirtualBox.



Quick Comparison

Architecture	Key Point
Monolithic	All in kernel, fast but hard to maintain
Layered	Structured in layers, easy to design
Microkernel	Minimal kernel, more secure
Hybrid	Combination of both
Virtual Machine	OS runs on top of virtual machine

2. Operating System Services

An OS provides various services to users and programs. These services make the computer system convenient and efficient.

Major OS Services :

- ① Program Execution - Loads program into memory, executes and terminates.
- ② I/O Operations - Provides uniform interface for I/O devices.
- ③ File System Manipulation - Create, delete, read, write, rename files and directories.
- ④ Communication - Allows process to exchange information (IPC - Inter Process Communication).
- ⑤ Error Detection - Detects and handles errors in CPU, memory, I/O devices.

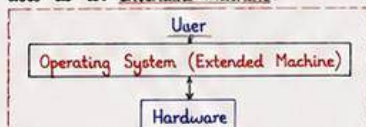
- ⑥ Resource Allocation - Allocates CPU, memory, I/O devices among processes.
- ⑦ Accounting - Keeps track of resource usage (CPU time, memory used, number of logins, etc.).
- ⑧ Protection & Security - Protects data and programs from unauthorized access.
- ⑨ User Interface - Provides GUI, CLI, Shell for user interaction.
- ⑩ System Calls - Interface between user programs and OS kernel.



3. OS as Extended Machine

OS hides the complexity of hardware and provides easy-to-use services.

It acts as an extended machine.



Benefits :

- Easy to use
- Efficient resource utilization
- Better performance
- Reliability and protection
- Supports growth and new technology

Remember

- ★ OS is a resource manager.
- ★ OS provides services to users and programs.
- ★ Architecture decides performance, security and reliability.
- ★ Good OS makes the system convenient, efficient and user friendly.



Viva Questions :

- Q1. What are the different architectures of Operating System ?
- Q2. What are the services provided by Operating System ?
- Q3. Why is OS called an extended machine ?

Architecture decides how OS works behind the scenes.



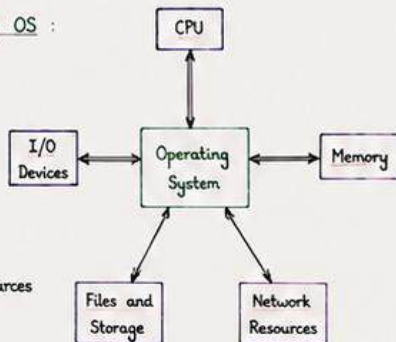
Page 6 : Architecture of Operating Systems & OS Services (Part 2)

4. OS as a Resource Manager

An OS manages all resources in a computer system efficiently and fairly.

Resources Managed by OS :

- ① CPU (Processor)
- ② Main Memory
- ③ I/O Devices
- ④ Files and Storage
- ⑤ Network Resources
- ⑥ Other Software Resources



Goal of OS as Resource Manager :

- Efficient resource utilization
- Fair allocation among users and programs
- Prevent resource conflicts
- Maximize throughput and performance

5. User View of OS

Users interact with OS through various interfaces.

Types of User Interfaces :

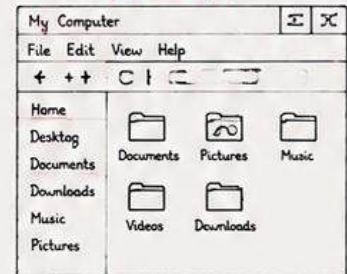
- Command Line Interface (CLI)
Example : Shell (Bash, PowerShell, CMD)
- Graphical User Interface (GUI)
Example : Windows Explorer, macOS Finder, Linux GUI
- Touch / Voice Interface in mobile and smart devices

CLI Example

```

$ ls -l
total 5
drwxr-x-x 2 user user 4096 May 20
-rw-r--r- 1 user user 1234 May 20
$ _
    
```

GUI Example

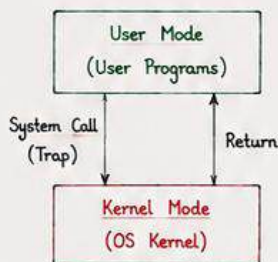


Note : GUI is more user friendly; CLI is powerful for advanced users.

6. Modes of OS Operation

OS operates in two modes :

- ① User Mode
 - Limited access to resources.
 - User programs run in this mode.
- ② Kernel Mode (Supervisor Mode)
 - Full access to all resources.
 - OS kernel runs in this mode.



7. System Calls

System call is the interface between a user program and the OS kernel.

Categories of System Calls :

- ① Process Control
(create, terminate, load, execute)
- ② File Management
(open, close, read, write, delete)
- ③ Device Management
(request device, release device, read, write)
- ④ Information Maintenance
(get time, get date, get system data)
- ⑤ Communication
(send message, receive message)
- ⑥ Protection
(set permissions, access control)

8. OS Design Goals

- ① Usability - Easy to use and learn.
- ② Efficiency - Efficient resource utilization and performance.
- ③ Reliability - System should be reliable and available.
- ④ Security - Protect data and resources from unauthorized access.
- ⑤ Flexibility - Easy to extend and modify.
- ⑥ Scalability - Should handle growth in users and resources.
- ⑦ Portability - Can run on different hardware platforms.

9. Summary Table

Aspect	Description	Examples
Architecture	Structure and organization of OS	Monolithic, Layered, Microkernel, Hybrid
Services	Services provided to users/programs	File Mgmt, I/O, Security, etc.
Resource Manager	Manages all system resources	CPU, Memory, Devices, Files, Network
User Interface	Way for users to interact with OS	CLI, GUI, Touch, Voice
Modes	Different operation modes	User Mode, Kernel Mode
System Calls	Interface to request OS services	open(), read(), fork(), exec()
Goals	Objectives of OS design	Usability, Efficiency, Reliability, etc.

Key Takeaways

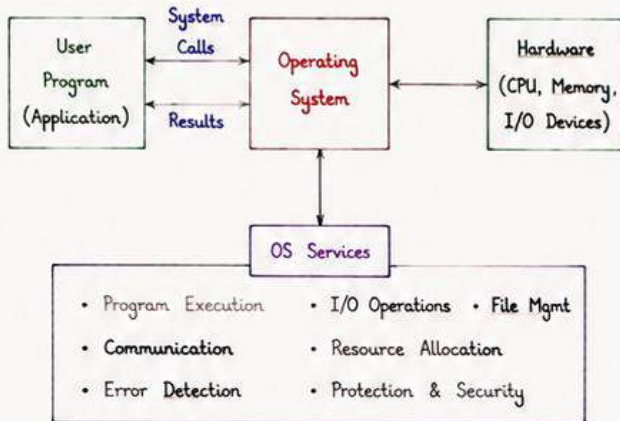
- ★ Architecture decides how OS is structured and operates.
- ★ OS provides essential services and manages all resources.
- ★ User interacts with OS through interfaces.
- ★ System calls allow user programs to request services from OS.
- ★ Good OS design achieves balance between performance, security and usability.



Page 7 : Architecture of Operating Systems & OS Services (Part 3)

1. Interaction of OS Components

All components of OS work together to provide services to users and programs.



Working :

- ① User program requests a service using system call.
- ② OS checks the request and performs required operation.
- ③ OS interacts with hardware to complete the task.
- ④ Result is returned back to the user program.

2. System Boot Process

The process of starting or booting a computer system.

Steps in Booting :

- ① **Power ON** : Power is supplied to the system.
- ② **BIOS/UEFI** : Performs POST (Power On Self Test) and initializes hardware.
- ③ **Boot Loader** : Loads the OS kernel into main memory.
- ④ **Kernel Initialization** : Kernel initializes device drivers, memory, processes, etc.
- ⑤ **System Services Start** : OS starts background services and system daemons.
- ⑥ **Login / User Interface** : User is presented with login screen or desktop environment.

★ The boot process ensures that OS and all necessary components are ready for user interaction.

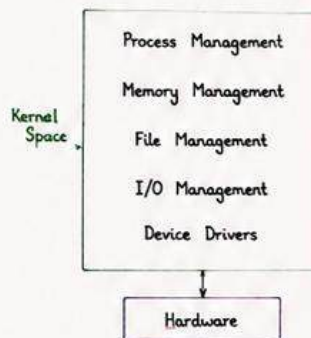
3. OS Services in Detail

- ① Program Execution : Loads a program into memory, runs it, handles termination and errors.
- ② I/O Operations : Provides uniform interface and handles all input/output operations.
- ③ File System Management : Organizes files and directories, provides operations on files.
- ④ Communication : Enables exchange of data between processes or systems.
- ⑤ Resource Allocation : Allocates CPU time, memory, I/O devices, and other resources fairly.
- ⑥ Protection & Security : Ensures data and resources are protected from unauthorized access.
- ⑦ Error Detection : Detects hardware and software errors and takes corrective actions.

4. Monolithic vs Layered OS Architecture

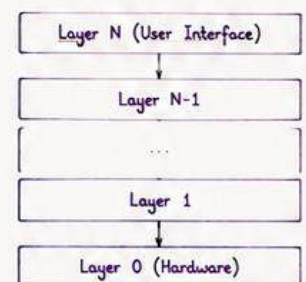
(A) Monolithic Architecture

- All OS services run in kernel space.
- Fast but hard to maintain and debug.



(B) Layered Architecture

- OS is divided into layers.
- Each layer uses services of lower layer.
- Easier to design and modify.



5. Summary

- ★ OS architecture decides how components and services are organized.
- ★ OS acts as a bridge between user and hardware.
- ★ It provides essential services for program execution and resource management.
- ★ Different architectures (Monolithic, Layered, Microkernel, Hybrid, Virtual Machine) are designed for different needs.
- ★ Good OS design ensures efficiency, security, reliability and user satisfaction. 😊

Viva Questions

- Q1. What is the role of OS as a resource manager?
- Q2. Differentiate between Monolithic and Layered architecture.
- Q3. List and explain the major OS services.
- Q4. What happens during system boot process?
- Q5. How users interact with OS?

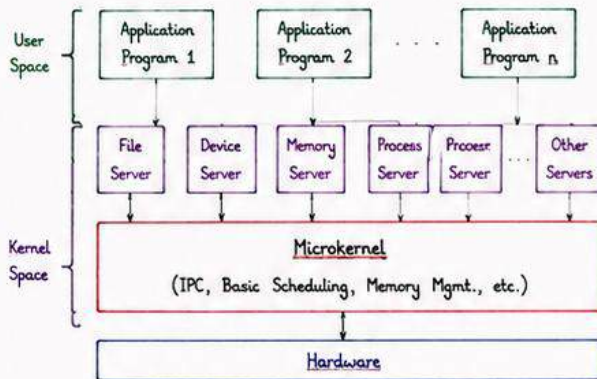


Page 8 : Architecture of Operating Systems & OS Services (Part 4)

1. Microkernel Architecture in Detail

Microkernel keeps only essential services in kernel space.

Other services run in user space as servers.



Advantages :

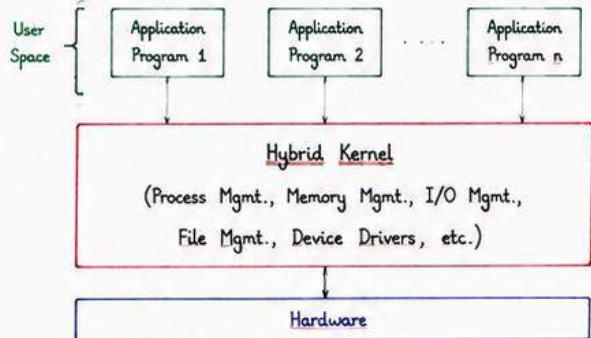
- ① More reliable (fault in a server doesn't crash whole system).
- ② Easier to extend with new services.
- ③ Better portability.
- ④ Improved security and stability.

2. Hybrid Kernel Architecture in Detail

Hybrid kernel combines advantages of both

Monolithic and Microkernel.

Many modern OS follow hybrid approach.

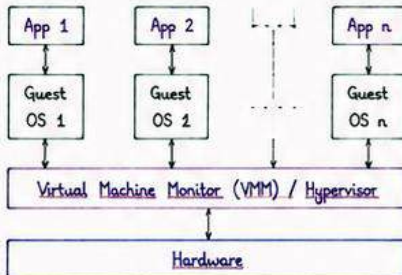


Features :

- Most services run in kernel space for speed.
- Some services can run in user space for flexibility.
- Balance between performance and reliability.
- Examples : Windows, macOS, Android, iOS.

3. Virtual Machine Architecture

- OS runs on top of a Virtual Machine Monitor (VMM).
- Multiple OSs can run on the same hardware.
- Used in virtualization and cloud computing.



Benefits :

- High hardware utilization.
- Strong isolation between OS instances.
- Easy to create and manage multiple environments.

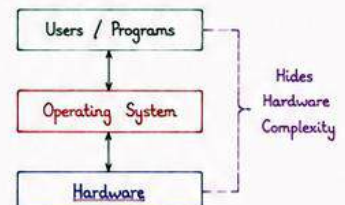
4. OS Services Classification

OS services can be classified as :

- ① User Oriented Services
e.g., User Interface, Program Execution, Communication.
- ② Process Oriented Services
e.g., Process Creation, Scheduling, Termination.
- ③ Storage Oriented Services
e.g., File Management, Secondary Storage Management.
- ④ Device Oriented Services
e.g., Device Management, I/O Operations.
- ⑤ Information Maintenance Services
e.g., Resource Allocation, Accounting, Protection & Security.

5. OS and Hardware Relationship

- OS acts as an intermediary between users/programs and hardware.
- It hides the complexity of hardware and provides easy-to-use services.



Key Point :

- ★ OS makes the computer system convenient to use and ensures efficient and safe execution.



Quick Comparison :

Architecture	Where Services Run	Performance	Reliability	Examples
Monolithic	All in Kernel Space	High	Low	UNIX (Old), Linux (Modules)
Layered	In Different Layers	Medium	Medium	THE, Educational OS
Microkernel	Essential in Kernel, Others in User Space	Medium	High	MINIX, GNX
Hybrid	Most in Kernel, Some in User Space	High	High	Windows, macOS, Android
Virtual Machine	Guest OS on VMM	Medium	High (Isolation)	VMware, VirtualBox

Viva Questions :

- Q1. What is Microkernel architecture ?
- Q2. Explain Hybrid Kernel architecture.
- Q3. How does OS act as a Resource Manager ?
- Q4. What are the major OS services ?
- Q5. How does Virtual Machine architecture work ?



★ A good OS is like a good manager - it makes everything work together smoothly!



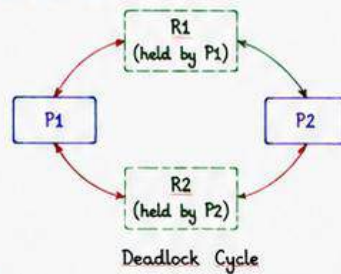
Page 9 : Miscellaneous & Additional Concepts (Part 1)

1. Deadlock

Deadlock is a situation where a set of processes are blocked because each process is waiting for a resource that is held by another process in the set.

Necessary Conditions :

- ① Mutual Exclusion
- ② Hold and Wait
- ③ No Preemption
- ④ Circular Wait

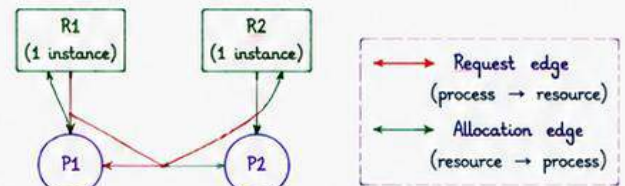


Prevention / Avoidance / Detection :

- Prevention : Break any one of the 4 conditions.
- Avoidance : Banker's Algorithm (safe state).
- Detection : Resource Allocation Graph, Recovery.

2. Resource Allocation Graph

A directed graph used to represent resource allocation and request relationships.



Used for :

- Deadlock detection (cycle means deadlock).
- Resource allocation analysis.
- Helps in understanding process-resource relationships.

3. Paging vs Segmentation

	Paging	Segmentation
Basic Unit	Fixed-size pages	Variable-size segments
Logical View	No (linear view)	Logical (user view)
Fragmentation	Internal (fragmentation)	External (fragmentation)
Address Format	Page number + offset	Segment number + offset
Table Size	Smaller	Larger
Use Case	Simple memory management	Better for modular programs

4. Thrashing

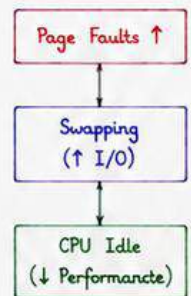
Thrashing is a condition where the system spends more time swapping pages than doing useful work.

Causes :

- Too few frames per process.
- High degree of multiprogramming.
- Poor page replacement algorithm.

Solutions :

- Increase number of frames.
- Use working set model.
- Reduce degree of multiprogramming.



5. Memory Management

It is responsible for allocating and deallocating memory to processes and managing virtual memory.

Functions :

- Allocation and deallocation.
- Address translation.
- Memory protection.
- Swapping (if needed).

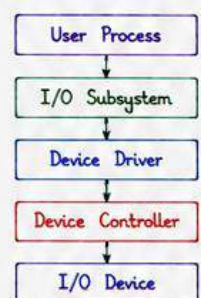


6. I/O Management

Handles communication between the system and external devices.

Components :

- Device Drivers
- I/O Subsystem
- Device Controllers
- I/O Requests & Queues

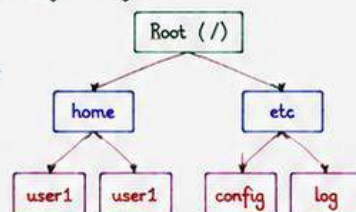


7. File System

Manages files and directories on secondary storage.

Functions :

- File creation, deletion, read, write.
- Directory management.
- File protection and access control.
- Free space management.

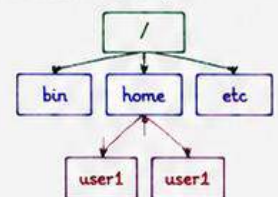


8. Directory Structure

Organizes files and subdirectories in a hierarchical form.

Types :

- Single Level Directory
- Two Level Directory
- Tree Structured Directory
- Acyclic Graph Directory (rare)



10. PROTECTION SYSTEM

1. Introduction

Protection in OS ensures that resources are accessed only by authorized users and programs.

It prevents unauthorized access, misuse, modification or destruction of data and system resources.

2. Need for Protection

- Multiple users share system resources.
- Prevents accidental or intentional damage.
- Protects data integrity and privacy.
- Ensures system reliability and security.
- Helps in enforcing access control policies.

3. Goals of Protection

- ① Ensure that all access to system resources is controlled.
- ② Specify the control over resources.
- ③ Provide convenient and easy mechanism for resource access.
- ④ Keep the performance overhead minimum.

4. Domain and Access Right

Domain : A collection of all subjects that have the same access rights.

Access Right : The permission to perform certain operations on objects.

Example Access Rights :

- Read (R)
- Write (W)
- Execute (X)
- Append (A)
- Delete (D)
- List (L)
- Own (O)

A subject can access an object only if it has the required access rights.

5. Access Matrix

Access Matrix is a 2-D table that specifies which subjects have which access rights to which objects.

Rows → Subjects (Domains)

Columns → Objects (Resources)

Example :

		Objects			
		File 1	File 2	File 3	Printer
Subjects	User 1	R, W	R	-	W
	User 2	R	R, W	R	-
	User 3	-	R	R, W	W

6. Implementation of Access Matrix

- Access matrix is too large and mostly sparse.
- Implemented using efficient data structures.

Two common methods :

① Access Control List (ACL)

For each object, maintain a list of subjects and their access rights.

② Capability List

For each subject, maintain a list of objects and their access rights.

7. Access Control

- Mechanism that regulates access to resources.
- Based on protection policies.

Types :

- ① Discretionary Access Control (DAC)
- ② Mandatory Access Control (MAC)
- ③ Role Based Access Control (RBAC)

8. Comparison of DAC and MAC

Aspect	Discretionary Access Control (DAC)	Mandatory Access Control (MAC)
Control	Object owner decides access rights.	System (admin) decides access rights.
Flexibility	More flexible.	Less flexible.
Security	Less secure.	More secure.
Based on	Identity of user.	Security level / classification.
Example	Windows File Permissions.	Military systems, SE Linux.
Change Rights	Can be changed by owner.	Cannot be changed by user.
Policy	Subject to owner discretion.	Subject to system policy.

Note : Protection ensures that the right person accesses the right resource in the right way.



11. SYSTEM CALLS

1. Introduction

System calls provide an interface between a running program (user mode) and the operating system (kernel mode). They allow user programs to request services from the OS safely and in a controlled manner.

2. User Mode vs Kernel Mode

- **User Mode** : Mode in which user programs execute.
- Limited access to system resources.
- **Kernel Mode** : Mode in which OS runs with full access to all hardware and resources.



Users cannot directly access hardware or kernel functions. They must use system calls.

3. System Call Mechanism

- ① A user program calls a library function (e.g., printf()).
- ② Library function makes a system call (trap instruction).
- ③ Control switches from user mode to kernel mode.
- ④ OS performs the requested service.
- ⑤ Control returns from kernel mode to user mode.



4. System Call Interface

- System calls can be made using high level languages such as C, C++ through libraries (e.g., stdio.h, unistd.h).
- The library hides the low level details from the programmer.

Example in C :

```

#include <stdio.h>
#include <unistd.h>
int main() {
    write(1, "Hello OS", 8); // write() is a system call
    return 0;
}
    
```

5. Types of System Calls

System calls are generally classified into the following categories :

Category	Description	Examples (Linux)
1. Process Control	Create, terminate, load, execute, wait, etc.	fork(), exec(), exit(), wait(), kill()
2. File Management	Create, delete, open, close, read, write files.	open(), close(), read(), write(), unlink()
3. Device Management	Request device, release device, read, write, etc.	ioctl(), read(), write(), open(), close()
4. Information Maintenance	Get and set time/date, system info, attributes.	getpid(), time(), gettimeofday(), uname()
5. Communication	Create, delete communication links, send, receive messages.	pipe(), socket(), send(), recv(), shmget()
6. Protection	Set or get file permissions and access rights.	chmod(), chown(), access(), umask()

6. Example System Calls in Linux

System Call

Purpose

- | | |
|------------|---|
| • fork() | Create a new process |
| • exec() | Replace process memory with a new program |
| • wait() | Wait for a child process to terminate |
| • open() | Open a file |
| • read() | Read data from file or device |
| • write() | Write data to file or device |
| • close() | Close a file or device |
| • pipe() | Create a pipe for communication |
| • exit() | Terminate the calling process |
| • getpid() | Get process id |

7. Key Points

- System calls provide controlled access to OS services.
- They protect the system from unauthorized access.
- They abstract hardware details from the user.
- System calls are essential for program execution and OS management.

System calls are the foundation of OS services. They provide a controlled way for user programs to access hardware and OS facilities.



12. PROCESSES - INTRODUCTION

1. What is a Program ?

- A program is a passive entity.
- It is a set of instructions stored in a file on the disk.
- It becomes active only when executed.
- Example : A C program stored in a file `hello.c` is a program.

2. What is a Process ?

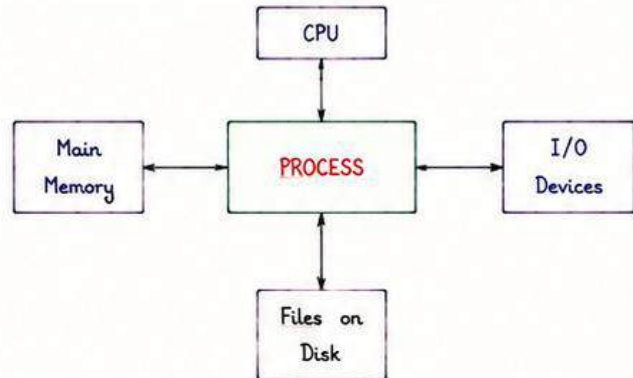
- A process is an active entity.
- It is a program in execution.
- When a program is loaded into memory and is being executed, it becomes a process.
- Each process has its own execution context, resources and state.
- Example : When `hello.c` is compiled and executed, it becomes a process.

3. Program vs Process

Program	Process
Passive (inactive)	Active (in execution)
Stored on disk	Resides in main memory
Single entity	Multiple instances possible
No resources allocated	Requires CPU, memory, I/O, etc.
Example : <code>hello.c</code>	Example : Running program

4. Process in the System

A process executes in the CPU by using system resources. The OS manages all the processes.



A process needs CPU time, memory, I/O devices and files to complete its execution.



5. Process Characteristics

- ① Has a unique Process ID (PID).
- ② Goes through various states during execution.
- ③ Requires resources for execution.
- ④ Can create new processes.
- ⑤ Terminates after completion.
- ⑥ Has a structure called Process Control Block (PCB).

6. Process Attributes

Each process has the following attributes :

<u>Process ID (PID)</u>	: Unique identifier assigned to the process.
<u>Process State</u>	: New, Ready, Running, Waiting, Terminated.
<u>Program Counter</u>	: Address of the next instruction to be executed.
<u>CPU Registers</u>	: Current values of CPU registers.
<u>CPU Scheduling Info</u>	: Priority, scheduling queue pointers, time slice, etc.
<u>Memory Information</u>	: Base and limit registers, page table / segment table.
<u>Accounting Information</u>	: CPU time used, elapsed time, user ID, job/account number, etc.
<u>I/O Status Information</u>	: List of I/O devices allocated to the process.
<u>Files</u>	: List of files opened by the process.
<u>Other Information</u>	: Signals, parent process ID, process group, etc.

Important Point

A process is more than just a program code. It includes the current activity, state and all resources allocated by the OS.



7. Multiple Processes

- The OS allows multiple processes to be present in the system at the same time.
- Processes may be independent or cooperate with each other.
- The OS is responsible for creating, scheduling and terminating processes.

★ A program is a set of instructions, but a process is a program in execution with context and resources. ★

13. PROCESS CONTROL BLOCK (PCB)

1. Introduction

To manage a process, the OS keeps all the information about that process in a data structure called Process Control Block (PCB).

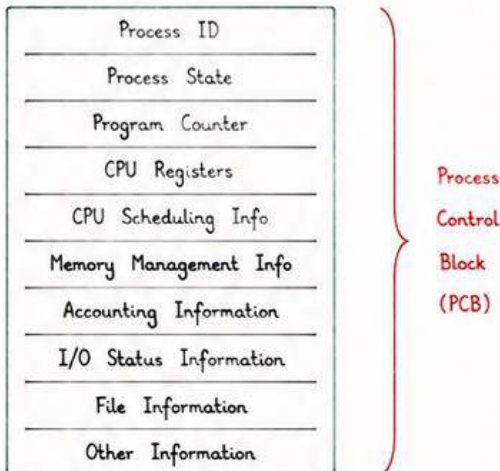
PCB is created when a process is created and destroyed when the process terminates.

PCB contains all the details and status information required by the OS to manage a process. 😊

2. Purpose of PCB

- It contains all information about a process.
- It helps the OS to manage and schedule the process.
- It is used during context switching.
- It helps in saving the state of a process.
- It is necessary for process synchronization and communication.

4. PCB Representation



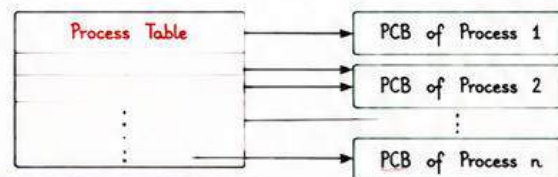
3. PCB Structure

A PCB consists of many fields. The exact contents may vary depending on the operating system.

Field Name	Description
Process ID (PID)	Unique identifier of the process.
Process State	Current state of the process (New, Ready, Running, Waiting, Terminated).
Program Counter	Address of the next instruction to be executed.
CPU Registers	Contents of all CPU registers associated with the process.
CPU Scheduling Info	Priority, queue pointers, time slice, scheduling parameters, etc.
Memory Management Info	Base and limit registers, page table pointer, segment table pointer, etc.
Accounting Information	CPU time used, elapsed time, user ID, job number, etc.
I/O Status Information	List of I/O devices allocated to the process.
File Information	List of files opened by the process.
Other Information	Signals, process group ID, parent process ID, etc.

5. PCB in Memory

- All PCBs are kept in a table called the Process Table.
- Each entry in the process table points to the PCB of a particular process.
- The OS uses PCB to perform context switching.
- When a process is switched out, its state is saved in its PCB.
- When it is switched in, the state is restored from its PCB.



6. Example

Suppose a process with PID = 101 is running. Its PCB may contain the following information :

Field	Example Value
PID	101
State	Running
Program Counter	0x0045A2
CPU Registers	R0 = 10, R1 = 20, R2 = 30, ...
CPU Scheduling Info	Priority = 2, Time Slice = 20ms
Memory Info	Base = 1000, Limit = 5000, Page Table Ptr = 0x7F00
Accounting Info	CPU Time = 120ms, User ID = 5, Job No = 3
I/O Status Info	Device = Printer (Busy), Device = Disk (Free)
File Info	File1.txt (Read), File2.c (Write)
Other Info	Parent PID = 90, Process Group = 2

Important Points

- ① PCB is unique for every process.
- ② Without PCB, the OS cannot manage multiple processes.
- ③ PCB plays an important role in scheduling and context switching.
- ④ PCB is kernel data structure and not accessible to user programs. 😊

14. PROCESS STATES

1. Introduction

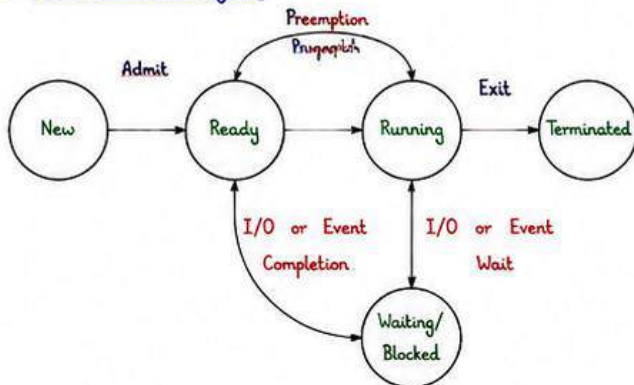
A process changes its state during its lifetime. The OS keeps track of each process state and moves it from one state to another based on events.

2. Process States

A process can be in one of the following states :

- ① **New** : The process is being created.
- ② **Ready** : The process is ready to run and is waiting for CPU.
- ③ **Running** : The process is currently executing on CPU.
- ④ **Waiting/Blocked** : The process is waiting for some event or I/O completion.
- ⑤ **Terminated** : The process has finished execution.

3. Process State Diagram



4. Description of States

- ① **New State** :
Process is being created. PCB is being initialized.
- ② **Ready State** :
All resources are allocated except CPU.
It is waiting in the ready queue.
- ③ **Running State** :
The process is currently executing instructions on CPU.
- ④ **Waiting/Blocked State** :
The process is waiting for an event such as I/O completion.
It cannot proceed until the event occurs.
- ⑤ **Terminated State** :
The process has completed execution or was aborted.
PCB is removed from the system.

5. State Transitions

- **New → Ready** : Admitted by long-term scheduler.
- **Ready → Running** : Selected by short-term scheduler (CPU dispatcher).
- **Running → Ready** : Preempted due to time quantum expiry or higher priority.
- **Running → Waiting/Blocked** : Requests I/O or waits for an event.
- **Waiting/Blocked → Ready** : I/O or event completion.
- **Running → Terminated** : Process completes execution or is aborted.

6. State Transition Table

From State	To State	Event/Condition	Action Taken
New	Ready	Process Admitted	PCB created, added to Ready Queue
Ready	Running	CPU Assigned	Scheduler dispatches
Running	Ready	Time Quantum Expires / Preemption	Put back in Ready Queue
Running	Waiting/Blocked	I/O Request / Event Wait	Move to Waiting Queue
Waiting/Blocked	Ready	I/O or Event Completion	Move to Ready Queue
Running	Terminated	Normal Completion / Abort	Remove PCB and resources released

7. Important Points

- A process changes state many times during its life.
- Only one process can be in Running state at a time in a single CPU system.
- **Ready Queue** contains all processes ready to run.
- **Waiting Queue** contains processes waiting for some event.
- OS uses these states to manage CPU and resources efficiently.

The state of a process changes due to events, and the OS is responsible for managing these state transitions.



15. PROCESS MANAGEMENT - OPERATIONS

1. Introduction

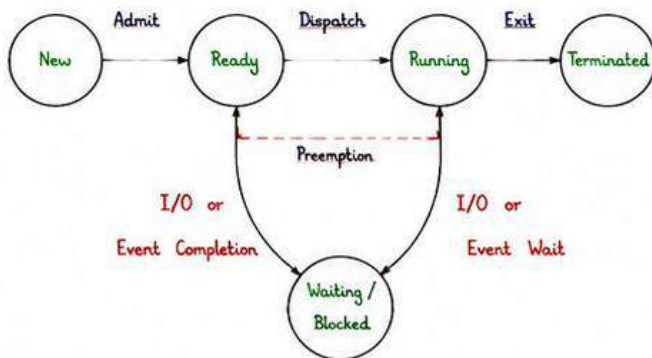
Process management is the set of activities performed by the OS to create, control, coordinate, synchronize and terminate processes.

Definition (Process Management) :-

Process management is the function of the operating system that handles the life cycle of a process from creation to termination and allocates the necessary resources for its execution. 😊

2. Life Cycle of a Process

A process goes through a life cycle consisting of several operations.



3. Process Operations

The OS performs the following main operations on a process during its life cycle.

S. No.	Operation	Description
1.	Process Creation	Creation of a new process.
2.	Process Scheduling	Selection of a process for CPU allocation.
3.	Process Execution	Execution of instructions by the CPU.
4.	Process Synchronization	Coordination of processes that share resources.
5.	Process Termination	Removal of the process from the system.

Important Points

- ★ A process is created, executed and terminated by the OS.
- ★ OS must ensure proper synchronization and communication.
- ★ OS must save and restore the context during state changes.
- ★ All resources allocated to a process must be released when it terminates. 😊

4. Process Creation

Definition :-

Process creation is the activity of creating a new process in the system.

Steps involved :-

- ① OS allocates a new PID to the process.
- ② Memory is allocated for the process.
- ③ PCB is created and initialized.
- ④ Necessary resources (files, I/O devices, etc.) are allocated.
- ⑤ The process is placed in the Ready Queue.

Methods of Process Creation :-

- System Initialization : OS creates many system processes.
- User Request : A user initiates a new process.
- Existing Process Creates : A process creates a child process.

Example in UNIX :-

fork() system call is used to create a new process. The created process is called the child process.

5. Process Termination

Definition :-

Process termination is the activity of ending a process and releasing all its resources back to the system.

Reasons for Termination :-

- Normal completion (exit).
- Error or exception.
- Fatal error.
- Killed by another process (abort).
- Time limit exceeded.

Steps in Process Termination :-

- ① OS performs termination system call.
- ② Parent process is informed (if any).
- ③ All resources allocated to the process are released.
- ④ PCB is deleted.
- ⑤ The process is removed from all queues.

Key Difference

Process Creation	Process Termination
Allocates resources	Releases resources
Creates PCB	Deletes PCB
Places in Ready Queue	Removes from all queues

16. CONTEXT SWITCHING

1. Introduction

Definition :

Context Switching is the process of saving the state of the currently running process and restoring the state of another process so that the CPU can be allocated to the new process.



- It is done by the Operating System.
- It occurs when the CPU is taken away from one process and given to another.
- It is an essential part of multitasking.

2. Why Context Switching is Required ?

- To allow multiple processes to share the CPU.
- To switch between user mode and kernel mode.
- To handle interrupts and I/O operations.
- To maintain fairness and proper scheduling.

3. When does Context Switching Occur ?

- When a time quantum expires.
- When a process goes to Waiting/Blocked state.
- When a higher priority process arrives.
- When an interrupt occurs.
- When the current process voluntarily yields the CPU.

4. What is Saved During a Context Switch ?

Item	Description
Program Counter (PC)	Address of next instruction.
CPU Registers	General purpose registers, accumulator, index registers, etc.
Stack Pointer (SP)	Top of the stack.
Process State	Running, Ready, Waiting, etc.
Memory Management Info	Page table base register, segment registers, etc.
I/O Status Info	List of allocated I/O devices.
Other Information	Priority, accounting info, signals, etc.

Note :

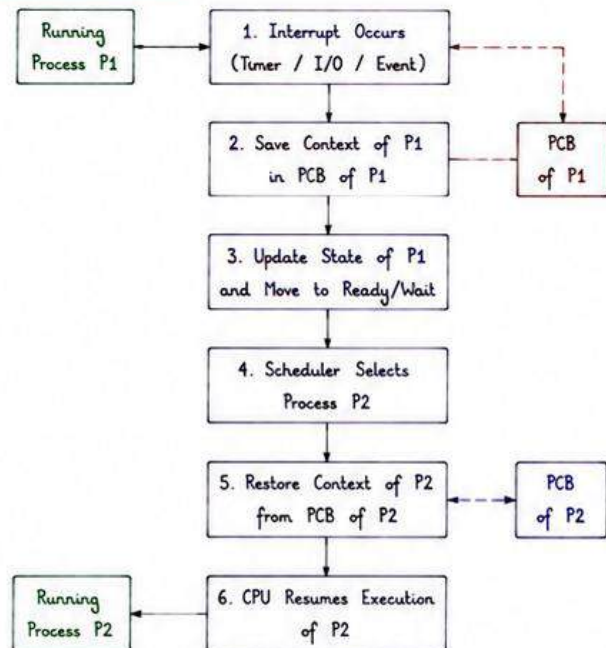
All this information is stored in the Process Control Block (PCB) of the process.



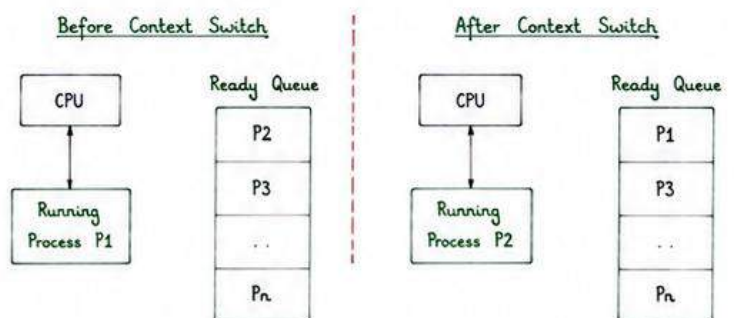
5. Steps in Context Switching

- ① Interrupt or event occurs (e.g., timer interrupt).
- ② CPU finishes the current instruction and enters kernel mode.
- ③ OS saves the state of the current process in its PCB.
- ④ OS updates the state of the current process (e.g., Running → Ready or Waiting).
- ⑤ Scheduler selects another process from the Ready Queue.
- ⑥ OS restores the state of the selected process from its PCB.
- ⑦ CPU resumes execution of the new process in user mode.

Context Switch Flow



6. Diagram - Before and After Context Switch



8. Important Points

- ★ Context switching is fully handled by the OS.
- ★ It occurs very frequently in multitasking systems.
- ★ It is expensive but necessary.
- ★ Efficient context switching improves system performance.
- ★ PCB plays a key role in context switching.

Context Switching is the backbone of multitasking. It allows the OS to give CPU time to multiple processes fairly.



17. INTERRUPTS

1. Introduction

An interrupt is a signal that causes the CPU to pause its current execution and transfer control to the Interrupt Service Routine (ISR) to handle an urgent event.

Definition :

An interrupt is a mechanism by which the operating system or hardware can get the attention of the CPU to handle high priority tasks.



2. Need for Interrupts

- To respond to I/O device requests.
- To handle hardware failures.
- To handle exceptions (e.g., divide by zero).
- To allow efficient CPU utilization.

3. Types of Interrupts

Interrupts are of two types :

- Hardware Interrupts
- Software Interrupts

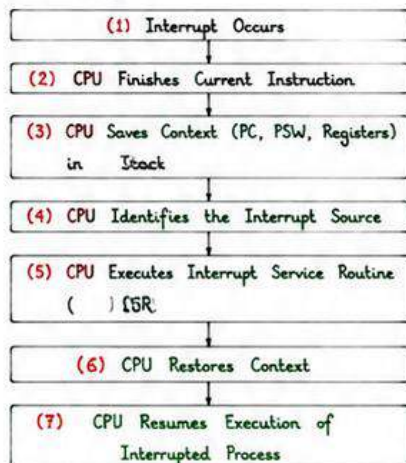
A. Hardware Interrupts

- Generated by external hardware devices.
- Examples : I/O device request, timer interrupt, power failure, machine check.
- They are asynchronous.

B. Software Interrupts

- Generated by executing a special instruction called system call or trap.
- Examples : System call, arithmetic overflow, illegal instruction, divide by zero.
- They are synchronous.

4. Interrupt Processing



5. Interrupt Vector Table (IVT)

The IVT contains the addresses of the ISR for different interrupts.

Interrupt No.	Address of ISR
0	Address of ISR0
1	Address of ISR1
2	Address of ISR2
.	.
n	Address of ISRn

6. Interrupt Handling Steps (Detailed)

- ① An interrupt occurs.
- ② CPU completes the current instruction.
- ③ CPU saves the current context (PC, PSW, Registers) in the stack.
- ④ CPU identifies the interrupt type using the interrupt vector.
- ⑤ Control is transferred to the ISR.
- ⑥ ISR handles the interrupt.
- ⑦ CPU restores the old context.
- ⑧ CPU resumes the execution of the interrupted process.

7. Maskable vs Non-maskable Interrupts

Maskable Interrupts	Non-maskable Interrupts
Can be ignored or disabled by the CPU.	Cannot be ignored or disabled.
Used for normal I/O operations.	Used for critical situations.
<u>Examples</u> : Timer interrupt, I/O interrupt.	<u>Examples</u> : Power failure, Machine check.
Controlled by Interrupt Enable (IF) flag.	Always have highest priority.

8. Important Points

- ★ Interrupts increase system efficiency.
- ★ Interrupts help in asynchronous communication.
- ★ An ISR should be short and fast.
- ★ Proper saving and restoring of context is essential.
- ★ Interrupts are the backbone of multitasking and real-time systems.

Note : Without interrupts, the CPU would have to continuously check devices (polling), which wastes CPU time. Interrupts make the system event-driven and efficient.



18. SYSTEM CALLS

1. Introduction

System calls provide an interface between a user program and the operating system kernel. They are the only way for a user program to request services from the OS.

Definition :

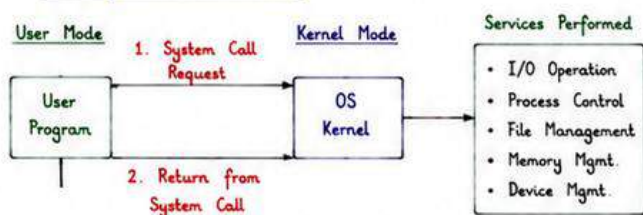
A system call is a controlled entry point provided by the OS that allows a user-level program to request services from the kernel.



2. Why System Calls are Needed ?

- To access hardware devices safely.
- To perform I/O operations.
- To manage processes and memory.
- To maintain protection and security.
- To provide a standard interface to the OS.

3. How System Call Works ?



- ① User program calls a library function (e.g., read()).
- ② Library function places system call number and arguments in registers/stack.
- ③ A software interrupt (trap) is executed.
- ④ CPU switches to kernel mode and control goes to OS.
- ⑤ OS performs the requested service.
- ⑥ Result/return value is returned to user program.
- ⑦ CPU switches back to user mode.

4. Types of System Calls

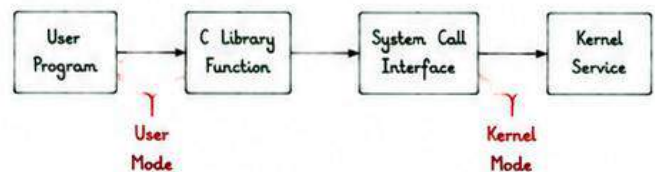
Type	Description
① Process Control	Create, terminate, load, execute, end, abort processes.
① File Management	Create, delete, open, close, read, write, reposition files.
③ Device Management	Request device, release device, read, write, get status, set attributes.
④ Information Maintenance	Get time, date, system data, process info, etc.
① Communication	Create, delete communication connection, send, receive messages.
④ Protection	Set permissions, get file/process attributes, access control.

5. Categories of System Calls (Examples)

Category	Examples
Process Control	fork(), exec(), exit(), wait(), kill()
File Management	open(), close(), read(), write(), lseek(), unlink()
Device Management	ioctl(), read(), write(), open(), close()
Information Maintenance	time(), getpid(), getuid(), gettimeofday()
Communication	socket(), connect(), send(), recv(), pipe()
Protection	chmod(), chown(), umask(), access()

6. System Call Interface

The system call interface is typically provided through a library (e.g., C standard library). The library hides the details of making a system call.



7. Example : Read System Call

```
int read(int fd, void *buf, size_t count);
```

Where,

- fd : File descriptor
 buf : Buffer where data will be stored
 count : Maximum number of bytes to read
 Return : Number of bytes actually read or -1 on error

8. Important Points

- ★ System calls cause a mode switch from user mode to kernel mode.
- ★ They are more time-consuming than normal function calls.
- ★ All system calls are executed in kernel mode.
- ★ System calls provide protection and virtualization.
- ★ They are the foundation for OS services.

Note : System calls are the bridge between user programs and the OS kernel. They allow programs to use hardware and OS services in a controlled, secure and efficient manner.



19. PROCESS COMMUNICATION (IPC)

1. Introduction

IPC (Inter-Process Communication) allows processes to exchange data and information and to synchronize their actions while executing.

Definition :

IPC is a set of mechanisms that allow processes to communicate and synchronize their execution in order to cooperate in a system. 😊

2. Need for IPC

- To exchange data and information.
- To synchronize and coordinate processes.
- To share resources.
- To speed up computation by cooperating.
- To support modularity in program design.

3. Models of Communication

A. Shared Memory Model



- A region of memory is shared by the communicating processes.
- The processes read/write data in this shared region.

B. Message Passing Model



- Processes communicate by sending and receiving messages.
- The kernel acts as an intermediary.

4. Mechanisms of IPC

Mechanism	Description
Pipes	Unidirectional communication between related processes.
Named Pipes	Like pipes, but can be used by unrelated processes.
Message Queues	Messages are sent/received through queues managed by the kernel.
Shared Memory	Processes share a common memory region.
Semaphores	Used for synchronization and mutual exclusion.
Sockets	Used for communication over a network.

5. Pipes (Example)

- A pipe is a unidirectional communication channel.
- It can be used only between related processes (parent-child).
- Data written to the pipe by one process can be read by the other.

Example System Call :

```
int fd[2];
pipe(fd); // fd[0] for reading, fd[1] for writing
```

6. Shared Memory (Example)

- A segment of memory is created and attached to the address space of communicating processes.
- Once created, both processes can read/write in it.

Example System Calls :

```
int shmid = shmget(key, size, IPC_CREAT | 0666);
char *ptr = (char *) shmat(shmid, NULL, 0);
shmdt(ptr);
shmctl(shmid, IPC_RMID, NULL);
```

7. Message Queues (Example)

- Messages are placed in a queue by the sender.
- The receiver retrieves them from the queue.
- Kernel ensures proper ordering and protection.

Example System Calls :

```
int msqid = msgget(key, IPC_CREAT | 0666);
msgsnd(msqid, &message, size, 0);
msgrcv(msqid, &message, size, type, 0);
msgctl(msqid, IPC_RMID, NULL);
```

8. Important Points

- ★ IPC allows cooperation between processes.
- ★ Shared memory is faster but needs synchronization.
- ★ Message passing is safer but slower.
- ★ Pipes are simple but limited to related processes.
- ★ Semaphores are used with IPC for synchronization.
- ★ Sockets enable IPC between processes on different systems.

Note : IPC is the key to cooperation and synchronization among processes. Proper IPC mechanism improves system performance, reliability and modularity. 😊

20. SYNCHRONIZATION AND MUTUAL EXCLUSION

1. Introduction

When multiple processes or threads access shared data and resources concurrently, the final outcome may be incorrect or inconsistent. Synchronization ensures correct cooperation among them, and mutual exclusion ensures that at most one process accesses a critical section at a time.

Definition :

Synchronization is the coordination of processes to ensure correct order of execution.

Mutual exclusion is a property that ensures only one process can enter its critical section at any given time. 😊

2. Need for Synchronization

- To maintain data consistency.
- To avoid race conditions.
- To coordinate access to shared resources.
- To ensure proper sequencing of events.
- To prevent deadlock and starvation.

3. Critical Section Problem

Each process has a segment of code that accesses shared resources. This segment is called the Critical Section.



Only one process should be in its critical section at a time.

4. Requirements for Critical Section Solution

- ① **Mutual Exclusion** : Only one process in CS at a time.
- ① **Progress** : If CS is free, one of the waiting processes must be selected.
- ③ **Bounded Waiting** : A bound exists on the number of times other processes can enter CS after a process has requested it.

5. Peterson's Solution (For Two Processes)

```
int flag[2] = {false, false};
int turn = 0;

Process Pi :
while (true) {
    flag[i] = true;
    turn = j; // give turn to other
    while (flag[j] && turn == j); // wait
    // Critical Section
    flag[i] = false; // Exit Section
    // Remainder Section
}
```

Where,

i, j are 0 or 1
and $i \neq j$

Note : Peterson's solution satisfies all three requirements.

6. Synchronization Hardware

Hardware support makes synchronization more efficient.

- **Test-and-Set** : Atomically tests a variable and sets it.
- **Swap Instruction** : Atomically swaps the contents of two memory locations.
- **Compare-and-Swap** : Compares a memory location with an expected value and replaces it if equal.

7. Semaphores

Definition :

A semaphore is a synchronization tool that uses an integer variable and two atomic operations wait() and signal().

Structure :

```
typedef struct {
    int count; // Semaphore value
    queue<int> Q; // Queue of waiting processes
} semaphore;
```

Operations :

wait(S) or P(S)

```
S.count--;
if (S.count < 0) {
    add this process to S.Q;
    block();
}
```

signal(S) or V(S)

```
S.count++;
if (S.count <= 0) {
    remove a process from S.Q;
    wakeup(process);
}
```

- If count $\geq 0 \rightarrow$ No process is waiting.
- If count $< 0 \rightarrow |count|$ represents number of waiting processes.

8. Types of Semaphores

Type	Description	Usage
Binary Semaphore	Value is 0 or 1 only.	Used for mutual exclusion.
Counting Semaphore	Value can range over an unrestricted domain.	Used for resource counting.

9. Classic Synchronization Problems

Problem	Objective
Producer - Consumer	Synchronize producer and consumer using a buffer.
Readers - Writers	Allow multiple readers or one writer at a time.
Dining Philosophers	Avoid deadlock while picking forks.

10. Important Points

- ★ Mutual exclusion prevents race conditions.
- ★ Synchronization ensures correct ordering of events.
- ★ Semaphores are powerful but must be used carefully.
- ★ Hardware support improves efficiency.
- ★ Proper synchronization increases performance and reliability.

★ Synchronization is the key to correct and efficient execution of concurrent processes. ★

21. DEADLOCKS

1. Introduction

A deadlock is a situation where a set of processes are blocked forever because each process is waiting for a resource that is held by another process in the set.

Definition :

Deadlock is a state where a set of processes are waiting for each other to release resources, and none of them can proceed.



2. Necessary Conditions for Deadlock (Coffman Conditions)

Deadlock can occur only if all the following four conditions hold simultaneously.

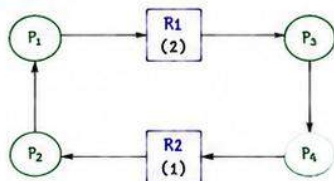
- ① **Mutual Exclusion** : Resources cannot be shared.
- ② **Hold and Wait** : A process holds at least one resource and waits for others.
- ③ **No Preemption** : Resources cannot be forcibly taken away; they can be released only voluntarily.
- ④ **Circular Wait** : A set of processes exists such that each process is waiting for a resource held by the next process in the set.

3. Resource Allocation Graph (RAG)

RAG is a directed graph used to represent the allocation of resources to processes and their requests.

- $P_i \rightarrow$ Process
- $R_j \rightarrow$ Resource Type (with instances)
- $P_i \rightarrow R_j$: Request edge
- $R_j \rightarrow P_i$: Assignment edge

Example :



If the graph contains a cycle and each resource type has only one instance, then deadlock exists.

4. Methods to Handle Deadlock

- **Prevention** : Ensure at least one of the four necessary conditions never holds.
- **Avoidance** : Grant resources only if the system remains in a safe state.
- **Detection & Recovery** : Allow deadlocks to occur, then detect and recover.
- **Ignore** : Used when deadlocks are rare and cost of prevention is high.

5. Deadlock Prevention

Break at least one of the four necessary conditions.

Condition	Prevention Technique
Mutual Exclusion	Make resources sharable where possible.
Hold and Wait	Require a process to request all resources at once or release all held resources before requesting new ones.
No Preemption	If a process holding resources requests another that is unavailable, preempt the held resources.
Circular Wait	Impose a total ordering of resource types and require processes to request in increasing order.

6. Deadlock Avoidance

- Requires prior knowledge of maximum resource needs.
- Allocate resources only if the resulting state is safe.
- Banker's Algorithm is the most common avoidance algorithm.

7. Deadlock Detection

- Allow deadlock to occur, then detect using:
- Resource Allocation Graph Algorithm (for single instance)
- Wait-For Graph Algorithm (for multiple instances)

8. Deadlock Recovery

Approach	Method
Process Termination	• Abort all deadlocked processes, or • Abort one process at a time until deadlock is removed.
Resource Preemption	• Preempt resources from some processes and allocate to others. • Select victim process based on cost factor (priority, resources used, time to completion, etc.).

9. Example - Deadlock (Single Instance)

R1 and R2 have one instance each.

Allocation		
Process	Holding	Requesting
P1	R1	R2
P2	R2	R1

P1 holds R1 and waits for R2.
P2 holds R2 and waits for R1.
Cycle exists → Deadlock.

10. Important Points

- ★ Deadlock leads to infinite waiting.
- ★ All four conditions must hold simultaneously.
- ★ Prevention is simple but may reduce resource utilization.
- ★ Avoidance needs more computation but is efficient.
- ★ Detection and recovery is flexible but costly.

Note : Deadlock handling is essential to keep the system reliable, efficient and responsive.



22. THREADS

1. Introduction

A thread (also called lightweight process) is the smallest unit of CPU execution within a process. Multiple threads of a process execute concurrently and share the same resources of that process.

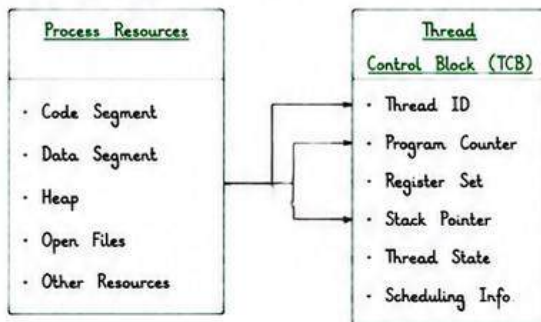
Definition :

A thread is a separate flow of control within a process. It has its own program counter, register set and stack, but shares the code, data and other resources of the owning process with other threads. 😊

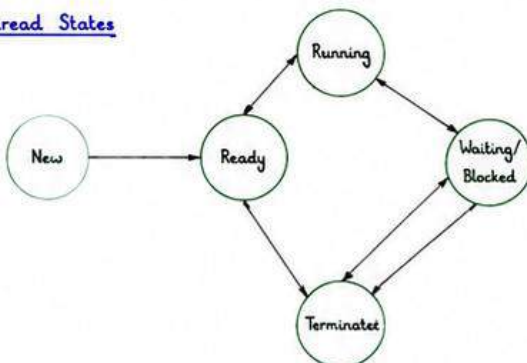
2. Need for Threads

- **Responsiveness** - One thread can continue even if another thread blocks.
- **Resource Sharing** - Threads of a process share memory and resources.
- **Economy** - Thread creation, termination and context switching are faster and require less overhead.
- **Scalability** - Better utilization of multi-processor/multi-core architectures.
- **Communication** : Easy communication between threads of the same process.

3. Thread Structure



4. Thread States



- **New** : Thread is being created.
- **Ready** : Ready to run and waiting for CPU.
- **Running** : Currently executing on the CPU.
- **Waiting/Blocked** : Waiting for event or resource.
- **Terminated** : Thread has finished execution.

5. Thread Operations

- ① **Thread Creation** - A new thread is created within a process.
- ② **Thread Termination** - A thread finishes execution and exits.
- ③ **Thread Scheduling** - The CPU is allocated to a thread.
- ④ **Thread Synchronization** - Mechanism to control access to shared resources.
- ⑤ **Context Switching** - CPU switches from one thread to another.

6. Benefits of Threads

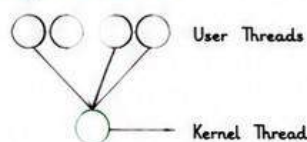
- Improved performance and throughput.
- Better CPU utilization.
- Reduced response time.
- Efficient communication.
- Parallelism within a single process.

7. Types of Threads

Type	Description
User-Level Threads	Managed by user-level library. Kernel is unaware of threads.
Kernel-Level Threads	Managed and supported by the OS kernel.
Hybrid Threads	Combination of user-level and kernel-level threads.

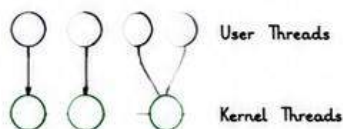
8. Thread Models

A. Many-to-One Model



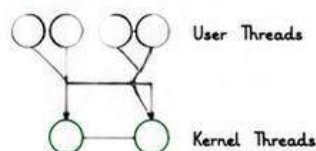
Many user-level threads mapped to one kernel thread. If one blocks, all block.

B. One-to-One Model



Each user thread mapped to a kernel thread. Allows true parallelism.

C. Many-to-Many Model



Many user threads mapped to many kernel threads. Combines benefits of both models.

9. Important Points

- ★ Threads share the code, data and resources of the process.
- ★ Each thread has its own stack and program counter.
- ★ Thread communication is easier than process communication.
- ★ Threads are suitable for multiprocessor systems.
- ★ Excessive number of threads may degrade performance.

23. PROCESSOR SCHEDULING - INTRODUCTION

1. Introduction

When multiple processes are ready to run and compete for the CPU, the OS must choose one of them and allocate the CPU to it. This task is called CPU scheduling.

Definition :

CPU scheduling is the activity of selecting one of the processes in the ready queue and allocating the CPU to it.



2. Need for CPU Scheduling

- To keep the CPU busy and utilized.
- To improve system throughput.
- To reduce waiting time for users.
- To ensure fairness among processes.
- To meet deadlines in real-time systems.

3. Levels of Scheduling

Level	Description
Long-Term (Job) Scheduler	Selects jobs from the new job pool and loads them into memory for execution.
Medium-Term Scheduler	Swaps processes in and out of memory to control the degree of multiprogramming.
Short-Term (CPU) Scheduler	Selects a process from the ready queue and allocates the CPU.

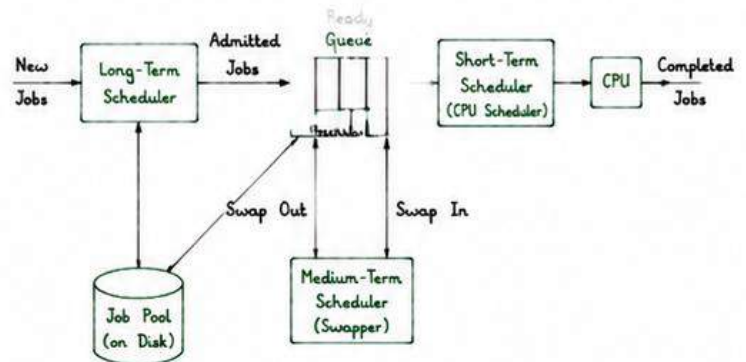
4. Scheduling Objectives

- Maximize CPU utilization
- Maximize throughput
- Minimize turnaround time
- Minimize waiting time
- Minimize response time
- Ensure fairness
- Meet deadlines (in real-time systems)

5. Scheduling Criteria

Criteria	Explanation
CPU Utilization	Percentage of time CPU is busy. Should be maximum.
Throughput	Number of processes completed per unit time. Should be maximum.
Turnaround Time	Total time taken from submission to completion. Should be minimum.
Waiting Time	Total time spent in ready queue. Should be minimum.
Response Time	Time from submission to first response. Should be minimum.

6. Scheduling Levels (Diagram)



7. Types of Scheduling

A. Preemptive Scheduling

- The running process can be interrupted and the CPU can be allocated to another process.

B. Non-Preemptive Scheduling

- Once the CPU is allocated to a process, it cannot be taken away until the process terminates or blocks.

8. Properties of a Good Scheduling Algorithm

- ① Should be fair.
- ② Should utilize the CPU effectively.
- ③ Should minimize waiting time and turnaround time.
- ④ Should ensure good response time.
- ⑤ Should avoid starvation.

9. Scheduling Trends

- Move from non-preemptive to preemptive.
- From static to dynamic scheduling.
- Priority-based and real-time scheduling for deadline systems.
- Fairness and quality of service in modern systems.

10. Important Points

- ★ CPU scheduling is done when a process changes state.
- ★ It is performed by the short-term scheduler.
- ★ It is a core part of process management.
- ★ Good scheduling improves performance and user satisfaction.
- ★ Different algorithms suit different environments.
- ★ No single algorithm is best for all systems.

Note : CPU scheduling is essential for efficient utilization of CPU and overall system performance. The choice of scheduling algorithm greatly affects system behavior.



24. SCHEDULING ALGORITHMS - PART 1

1. Introduction

A scheduling algorithm determines the order in which processes are selected from the ready queue and allocated the CPU.

Goals :

- Maximize CPU utilization
- Maximize throughput
- Minimize turnaround time
- Minimize waiting time
- Minimize response time
- Ensure fairness and avoid starvation.

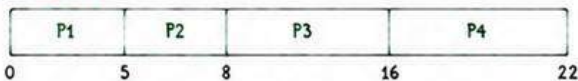
2. First Come First Serve (FCFS)

- Processes are executed in the order in which they arrive.
- Non-preemptive.
- Simple but may cause convoy effect.

Example :

Process	Arrival Time	Burst Time	Completion Time	Waiting Time	Turnaround Time
P1	0	5	5	0	5
P2	1	3	8	4	7
P3	2	8	16	6	14
P4	3	6	22	13	19

Gantt Chart :



$$\text{Average Waiting Time} = (0+4+6+13)/4 = 5.75$$

$$\text{Average Turnaround Time} = (5+7+14+19)/4 = 11.25$$

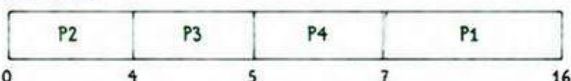
3. Shortest Job First (SJF)

- Selects the process with the smallest burst time.
- Can be preemptive or non-preemptive.
- Gives minimum average waiting time.

Example (Non-Preemptive) :

Process	Arrival Time	Burst Time	Completion Time	Waiting Time	Turnaround Time
P1	3	7	16	9	16
P2	1	3	4	0	3
P3	2	1	5	2	3
P4	3	2	7	2	4

Gantt Chart :



$$\text{Average Waiting Time} = (0+2+2+9)/4 = 3.25$$

$$\text{Average Turnaround Time} = (3+3+4+16)/4 = 6.5$$

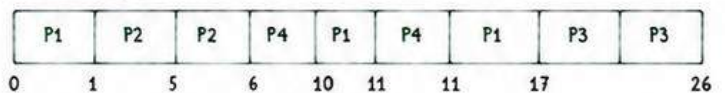
4. Shortest Remaining Time First (SRTF)

- Preemptive version of SJF.
- Always selects the process with the shortest remaining time.
- Better response time for short jobs.

Example :

Process	Arrival Time	Burst Time	Completion Time	Wait. Time	Turnaround Time
P1	0	8	17	9	17
P2	1	4	5	0	4
P3	2	9	26	15	24
P4	3	5	10	2	7

Gantt Chart :



$$\text{Average Waiting Time} = (9+0+15+2)/4 = 6.5$$

$$\text{Average Turnaround Time} = (17+4+24+7)/4 = 13$$

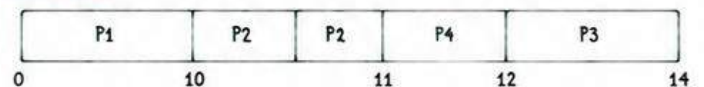
5. Priority Scheduling

- Each process is assigned a priority.
- Highest priority process is selected first.
- Can be preemptive or non-preemptive.
- Starvation may occur for low priority processes.

Example (Non-Preemptive, lower number = higher priority):

Process	Arrival Time	Burst Time	Priority	Completion Time	Waiting Time	Turnaround Time
P1	0	10	3	24	14	24
P2	1	1	1	2	0	1
P3	2	2	4	22	22	24
P4	3	1	3	3	0	1

Gantt Chart :



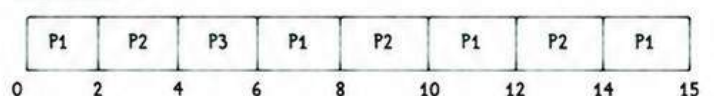
6. Round Robin (RR)

- Each process is executed for a fixed time quantum (q).
- After q time units, the process is preempted and moved to the end of the ready queue.

Example (q = 2) :

Processes : P1(5), P2(4), P3(2) (All arrive at time 0)

Gantt Chart :



Note :

- ★ FCFS is simple but may cause long waiting time.
- ★ SJF gives minimum waiting time but needs knowledge of burst time.

- ★ SRTF is best for response time.
- ★ Priority scheduling needs aging to prevent starvation.
- ★ Round Robin is ideal for time-sharing systems.



24. SCHEDULING ALGORITHMS - PART 2

1. Multilevel Queue Scheduling

- Ready queue is partitioned into multiple queues.
- Each queue has its own scheduling algorithm.
- Fixed priority between queues (higher priority queue executes first).

Example :

Queue	Priority	Scheduling Algorithm	Time Quantum
Q0 (System)	1 (Highest)	Round Robin	q = 4
Q1 (Interactive)	2	Round Robin	q = 8
Q2 (Batch)	3	FCFS	-

- A process enters the highest priority queue for which it is eligible.
- Lower priority queues get CPU only when higher queues are empty.

2. Multilevel Feedback Queue Scheduling

- A process can move between queues.
- Processes that use more CPU time are moved to lower priority queues.
- Processes that wait too long are moved to higher queues (aging).

Example :

Queue	Priority	Scheduling Algorithm	Time Quantum
Q0	1 (Highest)	Round Robin	q = 2
Q1	2	Round Robin	q = 4
Q2	3	Round Robin	q = 8
Q3	4 (Lowest)	FCFS	-

3. Comparison of Algorithms

Algorithm	Preemptive	Selection Criteria	Advantages	Disadvantages
FCFS	No	Arrival Time	Simple	Convoy Effect
SJF	No	Burst Time	Min. Avg. Waiting Time	Need burst time in advance
SRTF	Yes	Remaining Burst Time	Better response time	Starvation possible
Priority	Yes/No	Priority	Fast for high priority jobs	Starvation possible
Round Robin	Yes	Time Quantum	Fair, good response time	Context switch overhead
Multilevel Queue	Yes/No	Queue Priority	Good for mixed systems	Lower queues may starve
Multilevel Feedback Queue	Yes	Priority + Aging	Reduces starvation Better balance	Complex to implement

4. Demand Scheduling

- Processes are brought into memory only when they are needed.
- Reduces the degree of multiprogramming.
- Useful in systems with limited memory.

Example :

In a job pool, only a few jobs are loaded into memory. Another job is loaded only when one leaves the system.

5. Real-Time Scheduling

- Used in real-time systems where deadlines are critical.
- Two types :
 - ① Hard Real-Time : Missing a deadline is unacceptable.
 - ② Soft Real-Time : Missing a deadline is tolerable but undesirable.
- Algorithms : Rate Monotonic, Earliest Deadline First (EDF), Least Laxity First (LLF).

Example (Rate Monotonic) :

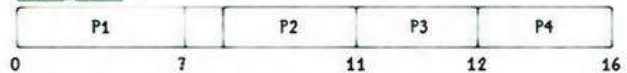
Task	Period	Execution Time	Priority
T1	4	1	1 (Highest)
T2	10	2	2
T3	20	4	3 (Lowest)

6. Summary of Algorithms with Example (Non-Preemptive unless specified)

Processes : P1(AT=0, BF=7), P2(AT=1, BT=4), P3(AT=2, BT=1), P4(AT=3, BT=4)

(A) FCFS

Gantt Chart :

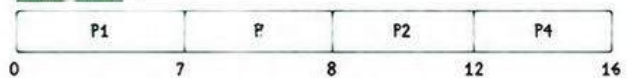


Average Waiting Time = $(0+7+11+12)/4 = 7.5$

Average Turnaround Time = $(7+8+12+16)/4 = 10.75$

(B) SJF (Non-Preemptive)

Gantt Chart :

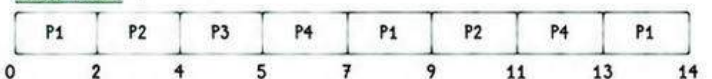


Average Waiting Time = $(0+7+8+12)/4 = 6.75$

Average Turnaround Time = $(7+8+9+13)/4 = 9.25$

(C) Round Robin (q = 2)

Gantt Chart :



Average Waiting Time = 4.25

Average Turnaround Time = 8.25

Note :

- No single algorithm is best for all situations.
- Choice of algorithm depends on system requirements.
- Preemptive algorithms give better response time.

- SJF/SRTF provide minimum average waiting time.
- Round Robin ensures fairness among all processes.



25. SCHEDULING ALGORITHMS - PART 3

1. Multilevel Queue Scheduling

- Ready queue is divided into separate queues.
- Each queue has a fixed priority.
- CPU is allocated to the highest priority non-empty queue.
- No process migration between queues.

Example :

Queue	Priority	Scheduling Algorithm	Time Quantum
Q0 (System)	1 (Highest)	Round Robin	q - 4
Q1 (Interactive)	2	Round Robin	q - 8
Q2 (User)	3	Round Robin	q - 16
Q3 (Batch)	4 (Lowest)	FCFS	-

Note : Higher priority queue must be empty before lower priority queue gets CPU.

2. Multilevel Feedback Queue Scheduling

- Processes can move between queues.
- If a process uses entire time quantum, it is moved to a lower priority queue.
- If it releases CPU before using full quantum, it may stay in the same queue or move to higher queue.
- Prevents starvation and adapts to process behavior.

Example :

Queue	Priority	Scheduling Algorithm	Time Quantum
Q0	1 (Highest)	Round Robin	q - 2
Q1	2	Round Robin	q - 4
Q2	3	Round Robin	q - 8
Q3	4 (Lowest)	FCFS	-

3. Comparison of Queue Based Algorithms

Algorithm	Process Migration	Priority Handling	Starvation Possible	Best Suitable For
Multilevel Queue	No	Fixed	Yes (Lower queues)	Systems with different process types
Multilevel Feedback Queue	Yes	Dynamic	Less (Almost No)	General purpose systems
Round Robin	No	Same	No	Time sharing systems

4. Important Points

- ★ Queue based algorithms are useful in multiprogramming systems.
- ★ Feedback queues adapt to process behavior dynamically.
- ★ Multilevel queue is simple but may cause starvation.
- ★ Multilevel feedback queue combines priority and fairness.

Note : Scheduling is the heart of process management.

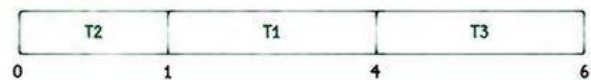
A good scheduling algorithm improves performance, fairness and user satisfaction.

5. Real-Time Scheduling (Contd.)

Example (Earliest Deadline First)

Task	Arrival Time	Deadline	Execution Time	Priority (Deadline Order)
T1	0	10	3	2
T2	0	5	1	1
T3	0	15	2	3

Gantt Chart :



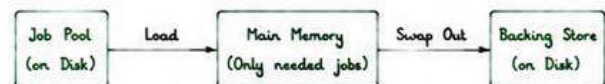
6. Real-Time vs General Purpose Scheduling

Feature	Real-Time Scheduling	General Purpose Scheduling
Goal	Meet deadlines	Maximize CPU utilization
Focus	Predictability	Throughput & fairness
Missed Deadline	Unacceptable	Acceptable
Algorithms	EDF, LLF, RM, DM	FCFS, SJF, RR, Priority
Environment	Real-time systems	General systems

7. Demand Scheduling (Swapping)

- Processes are brought into memory only when they are needed.
- If no free memory, a process is swapped out to disk.
- Decreases degree of multiprogramming.
- Useful in systems with limited main memory.

Example :



8. Summary of Scheduling Algorithms

Algorithm	Type	Preemptive	Best For	Key Advantage
FCFS	Non-Preemptive	No	Batch systems	Simple
SJF	Non-Preemptive	No	Batch systems	Min. Avg. Waiting Time
SRTF	Preemptive	Yes	Time critical jobs	Better Response Time
Priority	Preemptive/Non-Preemptive	Yes/No	Priority based systems	Handles important jobs
Round Robin	Preemptive	Yes	Time sharing systems	Fairness
Multilevel Queue	Preemptive	Yes	Systems with class separation	Simple & Effective
Multilevel Feedback Queue	Preemptive	Yes	General purpose systems	Adaptive & Fair

26. SCHEDULING ALGORITHMS - PART 4

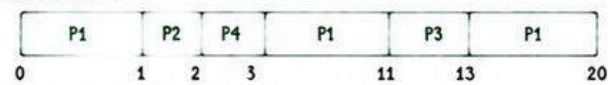
1. Priority Scheduling (Preemptive)

- The CPU is allocated to the highest priority process.
- If a new process with higher priority arrives, the current process is preempted.
- Lower number means higher priority.

Example :

Process	Arrival Time	Burst Time	Priority
P1	0	10	3
P2	1	1	1
P3	2	2	4
P4	3	1	2

Gantt Chart :



Average Waiting Time = 6.25

Average Turnaround Time = 12.25

2. Multilevel Queue Scheduling

- Ready queue is partitioned into multiple queues.
- Each queue has its own scheduling algorithm.
- Fixed priority between queues.
- No process migration between queues.

Example :

Queue	Priority	Scheduling Algorithm	Example Processes
System Queue	1 (Highest)	FCFS	System processes
Interactive Queue	2	Round Robin	P1, P2, P3
Batch Queue	3 (Lowest)	FCFS	P4, P5, P6

Higher priority queue is served first.

3. Multilevel Feedback Queue Scheduling

- Process can move between queues.
- If a process uses more CPU time, it is moved to lower priority queue.
- If it uses less CPU time, it may remain or move to higher queue.

Example :

Queue	Priority	Time Quantum	Scheduling Algorithm
Q0	1 (Highest)	2	Round Robin
Q1	2	4	Round Robin
Q2	4 (Lowest)	8	FCFS

New process enters in Q0.

If it uses entire time quantum, it is moved to next lower queue.

4. Comparison of Priority based Algorithms

Algorithm	Preemptive	Starvation	Implementation	Best For
Priority (Non-Preemptive)	No	Yes (Low priority processes)	Simple	Batch systems
Priority (Preemptive)	Yes	Yes (Low priority processes)	Complex	Real-time systems

5. Real-Time Scheduling (Contd.)

- Algorithms used :
 - Rate Monotonic (RM)
 - Earliest Deadline First (EDF)
 - Least Laxity First (LLF)

Example (Least Laxity First) :

Task	Arrival Time	Burst Time	Deadline	Laxity (Deadline - Remaining)
T1	0	3	6	6
T2	1	1	4	3
T3	2	2	7	5

Scheduling Order : T2 → T1 → T3

(At each point, task with least laxity is chosen)

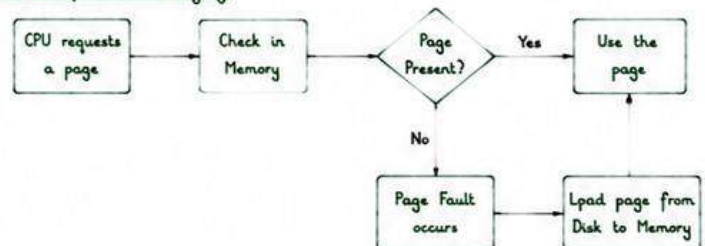
6. Real-Time Scheduling vs General Purpose Scheduling

Real-Time Scheduling	General Purpose Scheduling
Deadline is important	Deadline not important
Predictability is required	Predictability not required
Used in real-time systems	Used in general systems
Failure leads to system crash	Failure leads to performance loss
Examples : RM, EDF, LLF	Examples : FCFS, SJF, RR, etc.

7. Demand Paging (Virtual Memory Concept)

- Pages are brought into memory only when they are needed.
- If page not in memory → Page Fault occurs.
- Page is loaded from disk to memory.

Process of Demand Paging :



It reduces memory usage and improves efficiency.

8. Important Points

- ★ Lower priority number means higher priority.
- ★ Preemptive algorithms provide better response time.
- ★ Non-preemptive algorithms are simpler to implement.
- ★ Real-time scheduling is crucial for deadline based systems.
- ★ Demand paging improves memory utilization.
- ★ Multilevel feedback queue balances performance and fairness.

Note : Selection of algorithm depends on system requirements like performance, fairness, deadlines and nature of processes.

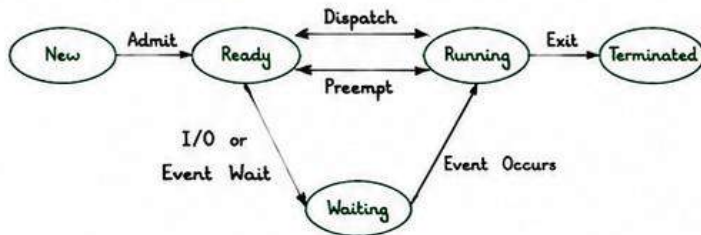


27. SUMMARY OF PROCESS MANAGEMENT & SCHEDULING

1. What we Studied ?

- Concepts of processes and their life cycle.
- Process states, PCB, context switching, interrupts.
- Inter-Process Communication and Synchronization.
- Threads (concept and models).
- Processor scheduling and different scheduling algorithms.

2. Process Life Cycle



3. Types of Scheduling

Type	Works On	Objective
Long-Term	Job pool	Controls degree of multiprogramming
Medium-Term	In-memory processes	Swaps processes (suspends/resumes)
Short-Term	Ready queue	Selects next process for CPU

4. Scheduling Criteria

Criteria	Goal
CPU Utilization	Maximum
Throughput	Maximum
Turnaround Time	Minimum
Waiting Time	Minimum
Response Time	Minimum

5. Common Scheduling Algorithms

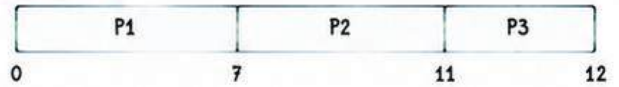
Algorithm	Preemptive	Key Feature	Best For
FCFS	No	Simple, fair	Batch systems
SJF	No	Minimum avg. waiting time	Batch systems
SRTF	Yes	Shortest remaining time	Time sharing
Priority	Yes/No	Handles important jobs	General
Round Robin	Yes	Time quantum	Time sharing
Multilevel Queue	Yes/No	Class based queues	Mixed systems
Multilevel Feedback Queue	Yes	Adaptive, prevents starvation	General

6. Algorithm Comparison Example

Processes : P1(AT=0, BT=7), P2(AT=2, BT=4), P3(AT=4, BT=1)

A) FCFS (Non-Preemptive)

Gantt Chart :



Average Waiting Time = $(0 + 5 + 7) / 3 = 4.00$

Average Turnaround Time = $(7 + 9 + 6) / 3 = 8.00$

(B) SJF (Non-Preemptive)

Gantt Chart :

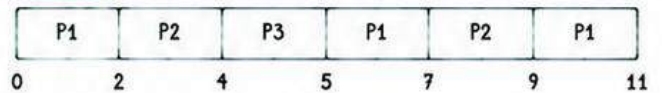


Average Waiting Time = $(0 + 3 + 6) / 3 = 3.00$

Average Turnaround Time = $(7 + 4 + 10) / 3 = 7.00$

(C) Round Robin (q = 2)

Gantt Chart :



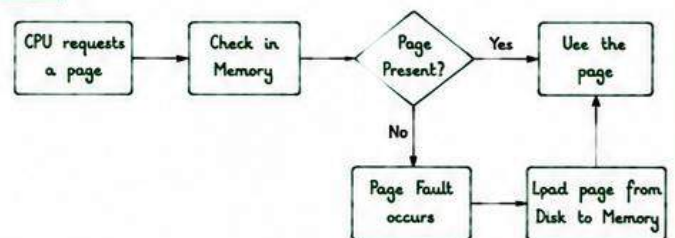
Average Waiting Time = 3.67

Average Turnaround Time = 7.67

7. Demand Paging

- Pages are loaded into memory only when they are needed.
- If page is not in memory → Page Fault occurs.
- Reduces memory usage and improves system efficiency.

Process :



8. Key Takeaways

- ★ CPU scheduling is the heart of process management.
- ★ Choice of algorithm depends on system requirements.
- ★ SJF/SRTF give minimum average waiting time.
- ★ Round Robin is ideal for time-sharing systems.
- ★ Priority scheduling handles important jobs but may starve low priority processes.
- ★ Demand paging and virtual memory improve memory utilization.
- ★ A good scheduling strategy improves performance, fairness and user satisfaction.

Note : Efficient process management and scheduling are essential for building high-performance, reliable and user-friendly operating systems.



28. MULTILEVEL FEEDBACK QUEUE SCHEDULING

1. Introduction

- Multilevel Feedback Queue (MLFQ) is an extension of multilevel queue scheduling.
- It allows a process to move between queues.
- CPU bound processes gradually move to lower priority queues.
- I/O bound processes stay in higher priority queues.
- Goal is to provide good response time and high CPU utilization.

2. Structure of MLFQ

Queue	Priority	Time Quantum	Scheduling
Q0	Highest	2	Round Robin
Q1	Medium	4	Round Robin
Q2	Low	8	Round Robin

- New processes enter the highest priority queue (Q0).
- If a process uses entire time quantum, it is moved to next lower queue.
- If a process finishes or yields before time quantum expires, it remains in the same queue.
- Processes in lower queues get CPU only when higher queues are empty.

3. Rules of MLFQ

- Always schedule from the highest non-empty queue.
- Within a queue, use Round Robin.
- Process moves to lower queue if it uses full time quantum.
- Process can move to higher queue if it yields or waits for I/O.
- Starvation is avoided by periodic priority boost.

4. Example

Consider 4 processes arriving at time 0.

Process	Arrival Time	CPU Burst Time
P1	0	10
P2	0	4
P3	0	7
P4	0	3

Time Quantum : Q0 = 2, Q1 = 4, Q2 = 8

Gantt Chart :

P1	P2	P3	P4	P1	P2	P3	P1	P3	P1	
0	2	4	6	8	12	14	16	20	22	24

(Detailed Queue Transitions)

Time	Running Process	In Queue after Execution
0 - 2	P1 (Q0)	P1 → Q1
2 - 4	P2 (Q0)	P2 → Q1
4 - 6	P3 (Q0)	P3 → Q1
6 - 8	P4 (Q0)	P4 completes
8 - 12	P1 (Q1)	P1 → Q2
12 - 14	P2 (Q1)	P2 completes
14 - 18	P3 (Q1)	P3 → Q2
18 - 22	P1 (Q2)	P1 → Q2 (rem 2)
22 - 24	P3 (Q2)	P3 completes
24 - 26	P1 (Q2)	P1 completes

5. Performance Metrics

Completion Time (CT)

Process	Completion Time
P1	26
P2	14
P3	24
P4	8

Turnaround Time (TAT) = CT - AT

Process	Arrival Time	Completion Time	TAT
P1	0	26	26
P2	0	14	14
P3	0	24	24
P4	0	8	8

Waiting Time (WT) = TAT - Burst Time

Process	Burst Time	TAT	Waiting Time
P1	10	26	16
P2	4	14	10
P3	7	24	17
P4	3	8	5

Average Waiting Time = $(16 + 10 + 17 + 5) / 4 = 12$

Average Turnaround Time = $(26 + 14 + 24 + 8) / 4 = 18$

6. Advantages

- Good response time for interactive processes.
- Prevents starvation through priority boosting.
- High CPU utilization.
- Adapts to process behavior.

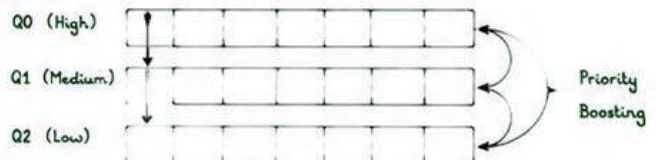
7. Disadvantages

- Complex to implement.
- Overhead of moving processes between queues.
- Performance depends on time quantum selection.

8. Priority Boosting

- To prevent starvation, all processes are moved to highest priority queue after a fixed time interval.
- This ensures fairness.

Example :



9. Summary

- MLFQ uses multiple queues with different priorities.
- Processes can move between queues based on behavior.
- CPU bound processes go to lower queues.
- I/O bound processes stay in higher queues.
- Provides good balance between response time and throughput.



29. REAL TIME SCHEDULING

1. Introduction

- Real time systems have timing constraints.
- Correctness depends not only on logical result but also on time at which result is produced.
- Two types of real time tasks :
 - ① Hard Real Time : Missing deadline is unacceptable.
 - ② Soft Real Time : Missing deadline is tolerable.

2. Characteristics

Feature	Description
Determinism	System behavior must be predictable.
Timeliness	Results must be produced within deadlines.
Reliability	Failure may lead to catastrophic results.
Concurrency	Multiple tasks must be handled together.

3. Real Time Task Parameters

- Release Time (r_i) : Time at which task becomes ready.
- Execution Time (C_i) : CPU time required.
- Deadline (d_i) : Time by which task must complete.
- Period (T_i) : Time between two successive releases.

4. Scheduling Objectives

- Meet all deadlines.
- Maximize CPU utilization.
- Minimize response time and jitter.
- Provide fairness among tasks.

5. Types of Real Time Scheduling

Type	Description	Example
Hard Real Time Scheduling	All deadlines must be met. Missing a deadline is system failure.	Airbag system, Missile control, Medical devices
Soft Real Time Scheduling	Deadlines are important but occasional misses are acceptable.	Video conferencing, Multimedia systems, Online gaming

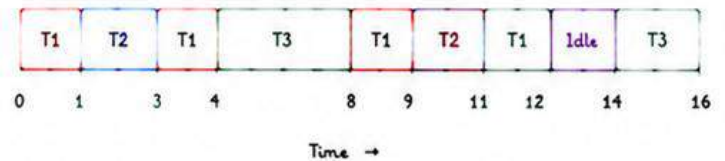
6. Real Time Scheduling Algorithms

- Rate Monotonic Scheduling (RMS)
 - Static priority based on period.
 - Shorter period $\rightarrow$ Higher priority.
- Earliest Deadline First (EDF)
 - Dynamic priority.
 - Task with nearest deadline gets highest priority.

7. Example : Rate Monotonic Scheduling

Task	Period (T_i)	Execution Time (C_i)	Priority
T1	10	1	1 (Highest)
T2	20	2	2
T3	40	4	3 (Lowest)

8. Gantt Chart (Rate Monotonic Scheduling)



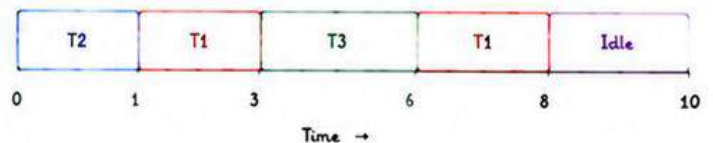
Task	Release Time	Execution Time	Period	Deadline
T1	0	1	10	10
T2	0	2	20	20
T3	0	4	40	40

All deadlines are met.

9. Example : Earliest Deadline First (EDF)

Task	Release Time	Execution Time	Deadline
T1	0	2	6
T2	0	1	4
T3	0	3	7

Gantt Chart (EDF)



All deadlines are met.

10. Comparison of RMS and EDF

Criteria	Rate Monotonic (RMS)	Earliest Deadline First (EDF)
Priority Basis	Fixed (based on period)	Dynamic (based on deadline)
Optimality	Not optimal	Optimal (for single processor)
Overhead	Low	High
∞	Hard real time systems	Hard & soft real time systems

11. Applications of Real Time Scheduling

- Industrial control systems
- Avionics and aerospace systems
- Medical monitoring systems
- Multimedia and online systems

12. Summary

- Real time scheduling ensures tasks meet timing constraints.
- Hard real time systems cannot miss deadlines.
- Soft real time systems allow occasional deadline misses.
- RMS is simple and efficient but not optimal.
- EDF is optimal but has higher overhead.



Note : Real time scheduling is crucial in systems where correctness depends on both result and time.



31. MODULE 1 - COMPLETE MIND MAP



FLOW OF OS CONCEPTS



KEY POINT

An Operating System acts as a Resource Manager, Process Manager and Service Provider. ☆

32. IMPORTANT DEFINITIONS & KEY TERMS

1	Operating System (OS) : System software that manages computer hardware and software resources and provides services to users and programs.	16	Response Time : The time from the submission of a request until the first response is produced.
2	Resource Manager : The OS acts as manager that allocates and controls resources like CPU, memory, I/O devices and files.	17	CPU Utilization : The percentage of time the CPU is busy executing processes.
3	System Call : Interface through which a user program requests services from the OS kernel.	18	Scheduling : The activity of selecting a process from the ready queue and allocating CPU to it.
4	Process : A program in execution. It is a unit of CPU utilization and resource ownership.	19	Preemptive Scheduling : A scheduling method where the CPU can be taken from a running process and given to another.
5	Process State : The current condition of a process (e.g., New, Ready, Running, Waiting, Terminated).	20	Non-Preemptive Scheduling : Once the CPU is allocated to a process, it continues until it releases the CPU or terminates.
6	PCB (Process Control Block) : Data structure that contains all information about a process.	21	FCFS : First Come First Serve scheduling algorithm. Process that arrives first gets the CPU first.
7	Context Switching : The mechanism of saving the state of the current process and restoring the state of another process.	22	SJF : Shortest Job First scheduling algorithm. Process with the smallest next CPU burst is chosen.
8	Interrupt : A signal that causes the CPU to stop its current execution and handle another task.	23	SRTF : Shortest Remaining Time First. Preemptive version of SJF.
9	IPC (Inter-Process Communication) : Mechanism that allows processes to communicate and synchronize their actions.	24	Priority Scheduling : Process with the highest priority is executed first.
10	Thread : The smallest unit of CPU execution within a process.	25	Round Robin (RR) : Each process gets the CPU for a fixed time quantum in cyclic order.
11	Multithreading : Allowing multiple threads to exist within a single process.	26	Multilevel Queue : Ready queue is divided into multiple queues with fixed priority between queues.
12	CPU Scheduler : The OS component that selects a process from the ready queue and allocates CPU to it.	27	Multilevel Feedback Queue : Processes can move between queues based on their behavior.
13	Throughput : The number of processes completed per unit time.	28	RMS (Rate Monotonic Scheduling) : Real-time scheduling using static priority based on period.
14	Turnaround Time : The total time taken from submission of a process to its completion.	29	EDF (Earliest Deadline First) : Real-time scheduling where process with earliest deadline gets CPU.
15	Waiting Time : The total time a process spends in the ready queue.	30	Real Time System : System in which results must be produced within strict time constraints.

QUICK RECAP



OS is the link between user and hardware.



It manages resources efficiently.



It provides services to users.



Scheduling decides who gets CPU and when.



Goal: Efficiency, Fairness and Reliability.

NOTE : Understanding these terms clearly is the base for solving any Operating System problem.



33. IMPORTANT QUESTIONS (EXAM + VIVA)

A. Very Short Answer Questions (1 Mark)

1. What is an Operating System ?
2. What is a Process ?
3. What is a Thread ?
4. What is a System Call ?
5. What is a PCB ?
6. What is Context Switching ?
7. What is a Process State ?
8. What is a System Call Interface ?
9. What is Preemptive Scheduling ?
10. What is Non-Preemptive Scheduling ?
11. What is Turnaround Time ?
12. What is Waiting Time ?
13. What is Throughput ?
14. What is FCFS Scheduling ?
15. What is Round Robin Scheduling ?

B. Short Answer Questions (2 Marks)

1. Write any two functions of an Operating System.
2. Differentiate between Process and Program.
3. List the different states of a process.
4. What is a PCB ? Write its contents.
5. What is Context Switching ? Why is it necessary ?
6. Differentiate between Preemptive and Non-Preemptive Scheduling.
7. What is the role of CPU Scheduler ?
8. What are System Calls ? Give examples.
9. What is the difference between Turnaround Time and Waiting Time ?
10. List any four types of Operating Systems.

C. Long Answer Questions (5 Marks)

1. Explain the need and functions of an Operating System.
2. Explain the Process States with a neat diagram.
3. Explain Process Creation and Termination in detail.
4. Explain the structure of PCB in detail.
5. Explain Context Switching with a neat diagram.
6. Explain the Scheduling Criteria in detail.
7. Explain FCFS Scheduling with example.
8. Explain Round Robin Scheduling with example.
9. Explain Multilevel Queue Scheduling.
10. Explain Real-time Scheduling with RMS and EDF.

D. Viva Questions (Oral)

1. What is the main purpose of an Operating System ?
2. How does an OS manage resources ?
3. What happens when a process goes from Running to Waiting state ?
4. Why do we need Context Switching ?
5. How does a Thread differ from a Process ?
6. What are the advantages of Multithreading ?
7. What is the difference between FCFS and SJF ?
8. Why is SRTF better than SJF in some cases ?
9. What is Priority Inversion ?
10. When will you use Round Robin Scheduling ?

E. Diagram-Based Questions

1. Draw and explain the Process State Diagram.
2. Draw the structure of Process Control Block (PCB).
3. Draw the Context Switching Diagram.
4. Draw the System Call Flow.

F. Match the Following

Column A	Column B
1. PCB	a) Shortest Job First
2. FCFS	b) Process Control Block
3. SJF	c) First Come First Serve
4. Round Robin	d) Interface to OS Services
5. System Call	e) Time Sharing Scheduling

Answer: 1-b, 2-c, 3-a, 4-e, 5-d

Tips for Exam

- Write definitions first.
- Use diagrams wherever possible.
- Write examples to support your answers.
- Highlight key points and headings.
- Manage time and attempt all questions.



OPERATING SYSTEMS - MODULE 1

Page No. Visual-1

VISUALISATION SHEET

1. TYPES OF OPERATING SYSTEMS

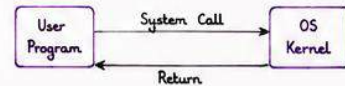
- Simple Batch System
- Multiprogramming Systems
- Time-Sharing System
- Personal Computer System
- Parallel System
- Distributed Systems
- Real-Time Systems



2. OS STRUCTURE

- System Components
 - CPU, Memory, I/O Devices, File System
- OS Services
 - Program Execution
 - I/O Operations
 - File Management
 - Communication
 - Error Detection
 - Resource Allocation
- System Calls
 - Interface between user programs and OS
- Protection System
 - Ensures security and safe access

3. SYSTEM CALLS



Types of System Calls

- Process Control
- File Management
- Device Management
- Information Maintenance
- Communication

4. PROCESSES

- Program in execution is called Process.
- Process States:



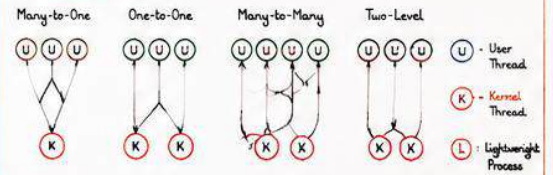
PCB (Process Control Block)

Field	Information
Process ID	Unique process identifier
Process State	Current state of process
Program Counter	Address of next instruction
CPU Registers	Values of CPU registers
CPU Scheduling Info	Priority, pointers, etc.
Memory Management Info	Base, limit, page tables, etc.
I/O Status Info	List of open files, I/O devices.

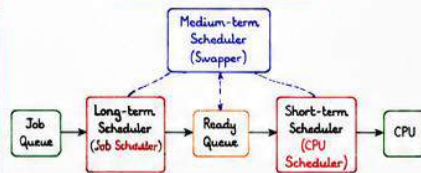
OPERATING SYSTEM

5. THREADS

- Thread is the smallest unit of CPU execution.
- Benefits:
 - Responsiveness
 - Resource Sharing
 - Economy
 - Scalability
- Thread Models:

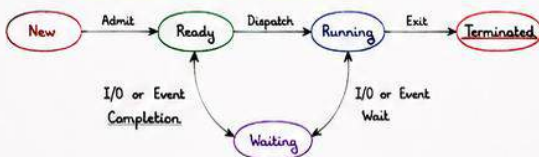


6. PROCESSOR SCHEDULING

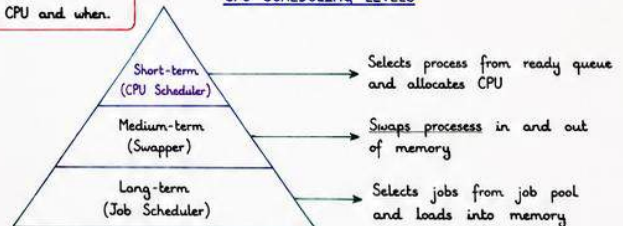


- Scheduling decides which process gets CPU and when.

PROCESS STATE DIAGRAM (DETAILED)



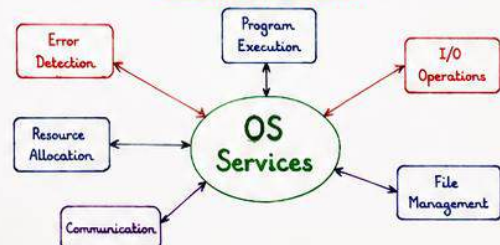
CPU SCHEDULING LEVELS



COMMON SCHEDULING ALGORITHMS AT A GLANCE

Algorithm	Type	Preemptive	Best For	Criteria / Basis
FCFS	Non-Preemptive	No	Simple, small number of processes	Arrival Time
SJF	Non-Preemptive	No	Min. Average Waiting Time	Burst Time
SRTF	Preemptive	Yes	Min. Average Waiting Time	Remaining Burst Time
Priority	Preemptive / Non	Yes / No	High Priority Tasks	Priority
Round Robin	Preemptive	Yes	Time Sharing (Fairness)	Time Quantum
Multilevel Queue	Preemptive	Yes	Different queues for different classes	Queue Priority
Multilevel Feedback Queue	Preemptive	Yes	Dynamic Priority Adjustment	Behavior
Real-Time (HMS)	Preemptive	Yes	Fixed Priority (Rate Monotonic)	Period
Real-Time (EDF)	Preemptive	Yes	Earliest Deadline First	Deadline

OS SERVICES OVERVIEW



Note: OS acts as an interface between user and hardware and manages all resources efficiently.